

AI OPERATIONS CONTROL PLANE

Master execution roadmap for a safe, evidence-driven Kubernetes operations investigator

401

EXECUTABLE TASKS

13

PHASES

526-845

FOCUSED HOURS

DAY 1

FIRST SLICE

START HERE

Use Section 12 and execute sessions T1-T7. The first target is one live, read-only broken-readiness investigation - not the year-one platform.

Operating rule: every new layer must improve the flagship investigation or make its quality measurable.

Source: Project Plan A | Execution model: solo maintainer using Codex locally, with human review and manual commits | Roadmap version 1.0

Contents

Navigation tip: use the PDF outline/bookmarks for major sections and phases. Every phase begins on a new page; every WBS task is formatted as a readable execution card.

1. Executive decision

4

2. Planning assumptions

4

3. Roadmap operating rules

5

4. Complete Work Breakdown Structure

7

Phase 0 - Scope and architecture foundation

7

Phase 1 - Repository and development foundation

11

Phase 2 - Same-day live walking skeleton

16

Phase 3 - Complete the reliable read-only Kubernetes vertical slice

23

Phase 4 - Reusable agent runtime and tool system

30

Phase 5 - Reproducible incident lab

36

Phase 6 - Evaluation framework

42

Phase 7 - Context engineering and operational knowledge

48

Phase 8 - Safe remediation

54

Phase 9 - External evidence integrations

60

Phase 10 - Service mode and engineer-facing experience

66

Phase 11 - Plugin and community foundation

74

Phase 12 - Release hardening, adoption proof, and 1.0 decision

79

5. Critical path and dependency graph

86

Primary dependency graph

86

Critical-path gates

87

Non-negotiable ordering constraints

87

6. Parallel work map

88

7. Milestones and timeline

89

Milestone definitions

89

Realistic timeline

89

8. Git commit strategy 91

Commit rules 91

Recommended Conventional Commit types 91

Suggested Day-0 commit sequence 91

9. Branch strategy 92

10. Documentation and planning structure 93

11. Daily execution mode 95

Standard 90-120 minute session 95

Canonical session queue 95

Session Definition of Done 96

12. First week - immediate checklist 97

Today: minimum working version 97

Days 2-7 98

Week 1 stop list 99

13. Ready-to-use Codex task packets for today 100

Packet A - Repository and minimal package foundation 100

Packet B - Reproducible broken-readiness scenario 100

Packet C - Typed read-only evidence collection 100

Packet D - One structured diagnosis and CLI 100

Packet E - Live verification and scope audit 101

14. Global Definition of Done 102

Source of truth: project plan A(9).txt **Planning horizon:** 6-12 months **Execution model:** solo maintainer using Codex locally, with the maintainer reviewing and committing every change **Roadmap version:** 1.0 - 2026-07-10

1. Executive decision

The smallest credible first version is not the control plane described in the year-one vision. It is one live, read-only, end-to-end investigation:

Given a deliberately unhealthy Kubernetes Deployment in a local cluster, a CLI collects real Kubernetes evidence, sends a bounded structured context to one model, and returns a schema-valid diagnosis that cites evidence identifiers and recommends a non-executed next action.

The first scenario is a **broken readiness probe**. The first version has one CLI, one cluster, one provider, three Kubernetes evidence collectors, one model decision, and one report. It has no web UI, API service, database, Prometheus, plugin system, remediation, long-term memory, multi-agent system, cloud deployment, or generic chat.

This is the target for today. Everything else waits until this path works.

Same-day acceptance gate

Today is successful only when all of the following are true:

- A public GitHub repository exists and the local repository is connected to it.
- A local kind cluster contains an intentionally unhealthy Deployment caused by a broken readiness probe.
- The CLI accepts a namespace and Deployment name.
- The system reads Deployment/Pod status, recent Kubernetes events, and recent container logs without invoking unrestricted shell commands through the model.
- Every observation receives a stable evidence ID.
- One structured model response names the likely cause, confidence, cited evidence IDs, uncertainty, and a recommended next action.
- The runtime rejects model-cited evidence IDs that were never collected.
- A deterministic test can run without a cluster or paid model call.
- A live run correctly diagnoses the known scenario.
- The sanitized trace and exact reproduction steps are documented.

If only the repository and architecture documents exist, the day failed. If the live diagnosis works but the repository is ugly, the day succeeded.

2. Planning assumptions

These choices fill gaps that the project specification intentionally leaves open. Record them as ADRs before implementation. They are planning defaults, not immutable product requirements.

Area	Working assumption	Revisit gate
Repository	Public monorepo named ai-ops-control-plane; Apache-2.0 license	Rename before the first public release if a better available name is selected
Language	Python 3.12+	Revisit only if measured runtime or contributor needs justify another language

Area	Working assumption	Revisit gate
Packaging	uv with one installable package initially	Revisit when the repository genuinely contains independently releasable packages
Schemas	Pydantic v2	Revisit only if the chosen model/provider interface makes it unsuitable
CLI	Typer	Revisit before declaring the CLI stable
Testing/quality	pytest, Ruff, and mypy	Revisit based on actual defect detection and contributor friction
Kubernetes access	Official Kubernetes Python client; no model-generated <code>kubectl</code> or shell	Do not relax this boundary
Local cluster	kind on Docker	Permit k3d/minikube later through documented adapters if users request it
Initial model	One OpenAI model with structured outputs, hidden behind a thin provider interface	Add a second provider only in Phase 4 after the runtime contract exists
Persistence	In-memory for the same-day slice; SQLite for local trace storage in Phase 4; PostgreSQL in service mode	Advance only when the preceding storage mode blocks a tested capability
Work rate	12-15 focused hours per week after the same-day sprint	Timeline moves proportionally if actual capacity differs
Security	Read-only Kubernetes credentials, secrets excluded from model context, tool output treated as untrusted	Required in every phase; never deferred
Versioning	0.x capability releases; no claim of production readiness without evidence	Consider 1.0 only after stable contracts, security review, and external-use proof

Estimation rules

- Estimates are focused implementation time for a developer who uses Codex but personally reviews the design, diff, and tests.
- Learning, installation failures, provider outages, and unfamiliar Kubernetes debugging can add 30-100%.
- A task estimated above two hours should be split across execution sessions even when it remains one coherent WBS deliverable.
- Phase dates are gates, not promises. Do not begin the next phase because the calendar says so; begin it because the preceding Definition of Done is true.

3. Roadmap operating rules

1. Keep work in progress at one behavior-changing branch.
2. Select the lowest-numbered unfinished task whose dependencies are complete.
3. Give Codex one bounded task, require repository inspection first, and forbid unrelated refactors.
4. Review the diff and understand every changed file before committing.
5. Run the narrow tests first, then the phase-level checks.
6. Commit a completed behavior with its tests and documentation.
7. Record newly discovered work in the backlog; do not silently expand the current task.
8. Stop a phase only at its explicit Definition of Done.

9. Do not create empty future architecture. Add a module when a current capability needs it.
 10. Every architectural layer must improve the flagship investigation or make its quality measurable.
-

4. Complete Work Breakdown Structure

Phase 0 - Scope and architecture foundation

Purpose: turn the project vision into one bounded, testable first workflow. **Estimated phase effort:** 6-10 hours. **Milestone:** M0 - architecture foundation.

Learning prerequisites

Know: basic Kubernetes Deployment/Pod/event/log concepts; the difference between observed evidence and inference; basic JSON schema concepts. Safely defer: agent frameworks, vector databases, multi-agent systems, cloud architecture, advanced threat modeling methods.

Tasks

0.1.1	Write the one-sentence product boundary	Easy / 15m
GOAL	State exactly what the project is and is not	
WHY	Prevent a generic “AI DevOps assistant” from replacing the actual product	
DEPENDS	None	DELIVERABLE docs/product/charter.md opening statement
ACCEPTANCE	It names an evidence-driven operations investigation runtime and excludes generic chat	

0.1.2	Define the flagship user and trigger	Easy / 15m
GOAL	Identify who starts the first investigation and with what input	
WHY	Inputs drive the CLI and state schema	
DEPENDS	0.1.1	DELIVERABLE User/trigger paragraph
ACCEPTANCE	It identifies an engineer, cluster context, namespace, workload kind/name, and symptom	

0.1.3	Freeze the first scenario	Easy / 10m
GOAL	Select broken readiness probe as the only same-day incident	
WHY	A single ground truth keeps implementation and verification bounded	
DEPENDS	0.1.2	DELIVERABLE Scenario scope statement
ACCEPTANCE	No second scenario or integration appears in the same-day scope	

0.1.4	Define the same-day input contract	Easy / 20m
GOAL	List the minimum fields required to start the investigation	
WHY	Prevent CLI and runtime ambiguity	
DEPENDS	0.1.3	DELIVERABLE Intake field list
ACCEPTANCE	Cluster context, namespace, workload type/name, symptom, and optional time window are covered	

0.1.5	Define the same-day final report contract	Easy / 25m
GOAL	Specify the fields the user must receive	
WHY	Makes the end-to-end result testable before implementation	
DEPENDS	0.1.4	DELIVERABLE Final diagnosis field list
ACCEPTANCE	Cause, confidence, evidence IDs, alternatives/uncertainty, and recommendation are mandatory	

0.1.6	Write explicit non-goals		Easy / 20m
GOAL	Lock out features that do not help today's investigation		
WHY	This project's primary risk is premature breadth		
DEPENDS	0.1.1	DELIVERABLE	docs/product/non-goals.md
ACCEPTANCE	Web, API, DB, Prometheus, write actions, memory, plugins, cloud, and multi-agent work are listed		

0.1.7	Define success and failure conditions		Easy / 20m
GOAL	Turn the first workflow into a binary gate		
WHY	"Looks promising" is not a release criterion		
DEPENDS	0.1.3-0.1.5	DELIVERABLE	Acceptance checklist
ACCEPTANCE	A stranger can determine pass/fail from observable behavior		

0.2.1	Draft InvestigationState v0		Medium / 45m
GOAL	Represent objective, phase, evidence, decision, budgets, and status		
WHY	Structured state is the product's core, not chat history		
DEPENDS	0.1.4-0.1.5	DELIVERABLE	docs/architecture/state-model.md
ACCEPTANCE	Every field has a name, purpose, type-level description, and lifecycle owner		

0.2.2	Draft EvidenceRecord v0		Medium / 30m
GOAL	Define stable evidence identity and provenance		
WHY	Diagnoses must cite concrete tool output		
DEPENDS	0.2.1	DELIVERABLE	Evidence record section
ACCEPTANCE	It includes ID, source tool, timestamp, target, observation, sensitivity, and raw-reference metadata		

0.2.3	Draft ToolResult v0		Medium / 40m
GOAL	Normalize success, partial result, and failure		
WHY	Kubernetes responses and errors need one deterministic envelope		
DEPENDS	0.2.2	DELIVERABLE	docs/architecture/tool-contract.md
ACCEPTANCE	Success flag, data/evidence, error category, retryability, duration, and truncation status are specified		

0.2.4	Draft FinalDiagnosis v0		Medium / 30m
GOAL	Define the schema the model must satisfy		
WHY	Structured output allows validation and evaluation		
DEPENDS	0.1.5, 0.2.2	DELIVERABLE	Diagnosis schema specification
ACCEPTANCE	All evidence references are IDs and unsupported IDs are invalid		

0.2.5	Draw the one-page first-workflow architecture	Medium / 45m
GOAL	Show intake -> collectors -> evidence -> model -> validator -> report	
WHY	Exposes vague or missing boundaries before coding	
DEPENDS	0.2.1-0.2.4	DELIVERABLE docs/architecture/overview.md diagram
ACCEPTANCE	Every component has an input/output and no future-only component is required	
0.2.6	Define read-only authorization boundary	Medium / 30m
GOAL	State exactly which Kubernetes operations are permitted	
WHY	Safety cannot depend on model compliance	
DEPENDS	0.2.3, 0.2.5	DELIVERABLE Security boundary section
ACCEPTANCE	Only get/list/watch-equivalent reads needed by the slice are permitted; mutation and shell are forbidden	
0.2.7	Create the initial threat model	Hard / 60m
GOAL	Identify credential leakage, prompt injection, oversized output, malicious logs, and over-permission	
WHY	Tool output is untrusted and credentials must stay outside prompts	
DEPENDS	0.2.5-0.2.6	DELIVERABLE docs/security/threat-model.md
ACCEPTANCE	Each threat has an asset, attack path, current mitigation, and deferred mitigation	
0.2.8	Specify the broken-readiness scenario	Medium / 45m
GOAL	Define initial state, injected fault, symptoms, ground truth, relevant evidence, distractions, expected diagnosis, and forbidden actions	
WHY	The scenario is both demo and future evaluation seed	
DEPENDS	0.1.3, 0.2.2	DELIVERABLE docs/scenarios/broken-readiness.md
ACCEPTANCE	Another developer can recreate the same fault without asking questions	
0.3.1	Record language and package-manager ADR	Easy / 30m
GOAL	Make the Python/uv choice explicit and reversible	
WHY	Hidden stack decisions create later churn	
DEPENDS	0.2.5	DELIVERABLE docs/adr/0001-language-and-packaging.md
ACCEPTANCE	Context, decision, alternatives, consequences, and revisit trigger are present	
0.3.2	Record monorepo and modular-monolith ADR	Easy / 25m
GOAL	Prevent premature repositories and services	
WHY	The specification explicitly requires a monorepo first	
DEPENDS	0.2.5	DELIVERABLE docs/adr/0002-monorepo.md
ACCEPTANCE	It permits logical modules while rejecting early microservices	

0.3.3	Record CLI-first ADR		Easy / 25m
GOAL	Freeze the initial interface and defer API/web		
WHY	CLI is the shortest path to the engineering audience		
DEPENDS	0.2.5	DELIVERABLE	docs/adr/0003-cli-first.md
ACCEPTANCE	It identifies the future review gate for service mode		

0.3.4	Create the deferred-capabilities backlog		Easy / 20m
GOAL	Preserve ideas without adding them to current scope		
WHY	Prevents fear of “losing” ideas from expanding the slice		
DEPENDS	0.1.6	DELIVERABLE	docs/planning/deferred.md
ACCEPTANCE	Every non-goal is recorded with a milestone, not an active task		

Phase 0 risks

Blocker or mistake	Typical beginner failure	Prevention
Architecture discussion never ends	Rewriting diagrams and schemas before any evidence is collected	Time-box Phase 0 to one working session plus review; schemas are explicitly v0
The first scenario quietly expands	Adding Prometheus, rollout diffing, API endpoints, or several failures	Treat 0.1.6 as a hard scope gate and move ideas to 0.3.4
Documentation becomes speculative	Describing components that have no same-day consumer	Delete any box not required by the first workflow
Safety is written as a prompt rule	Telling the model “do not mutate” without runtime/RBAC enforcement	Specify a read-only client and read-only Kubernetes identity

Phase 0 Definition of Done

- The entire first investigation fits on one page.
- Inputs, evidence records, tool results, final output, and read-only boundary are unambiguous.
- The first scenario has known ground truth and forbidden actions.
- Three ADRs and explicit non-goals exist.
- No future component is required to explain the first run.

Phase 1 - Repository and development foundation

Purpose: create the smallest reproducible repository and workstation needed for the first slice.

Estimated phase effort: 8-14 hours including installations. **Milestone:** repository initialized.

Learning prerequisites

Know: basic Git add/commit/push/branch operations; Python virtual environments; what Docker containers and kubeconfig are. Safely defer: advanced Git workflows, Kubernetes administration, packaging multiple distributions, release automation.

Tasks

1.1.1	Verify Git identity and GitHub authentication	Easy / 20m
GOAL	Ensure commits and pushes work before creating files	
WHY	Authentication failures should not block the first completed slice	
DEPENDS	Phase 0	DELIVERABLE Working Git name/email and GitHub auth
ACCEPTANCE	A read-only GitHub auth check succeeds and identity is correct	

1.1.2	Confirm the working repository name	Easy / 10m
GOAL	Use ai-ops-control-plane unless a collision exists	
WHY	Naming can consume hours without improving the product	
DEPENDS	1.1.1	DELIVERABLE Recorded working name
ACCEPTANCE	The name is available or one explicit fallback is selected	

1.1.3	Create the public GitHub repository	Easy / 10m
GOAL	Establish the remote source of truth	
WHY	The project is explicitly open source	
DEPENDS	1.1.2	DELIVERABLE Empty public GitHub repository
ACCEPTANCE	Repository URL opens publicly and contains no generated starter files that conflict with local setup	

1.1.4	Initialize the local repository	Easy / 15m
GOAL	Connect the empty local folder to Git	
WHY	Enables reviewable incremental work	
DEPENDS	1.1.3	DELIVERABLE Local main branch with origin
ACCEPTANCE	Branch is named main, remote is correct, and status is clean	

1.1.5	Add Apache-2.0 license	Easy / 10m
GOAL	Set the intended legal terms from day one	
WHY	Open-source contribution requires a clear license	
DEPENDS	1.1.4	DELIVERABLE LICENSE
ACCEPTANCE	GitHub recognizes Apache-2.0	

1.1.6	Add the minimum README		Easy / 25m
GOAL	State the honest current capability and status		
WHY	Prevents exaggerated claims and gives the repo an entry point		
DEPENDS	1.1.4, 0.1.1	DELIVERABLE	README.md
ACCEPTANCE	It has product statement, current status, non-goals link, and “not yet usable” notice		
1.1.7	Add ignore and text-normalization files		Easy / 25m
GOAL	Keep local state, secrets, caches, and OS noise out of Git		
WHY	Accidental secret/cache commits are avoidable		
DEPENDS	1.1.4	DELIVERABLE	.gitignore, .gitattributes, .editorconfig
ACCEPTANCE	Python, virtualenv, IDE, OS, coverage, build, .env, and kubeconfig artifacts are excluded		
1.1.8	Create only the needed initial directories		Easy / 20m
GOAL	Establish a minimal source/test/scenario/docs layout		
WHY	Avoid empty architecture for future phases		
DEPENDS	1.1.7	DELIVERABLE	src/, tests/, scenarios/broken-readiness/, and required docs/ paths
ACCEPTANCE	Every created directory has a same-week consumer		
1.1.9	Add the Phase 0 documents		Easy / 20m
GOAL	Place decisions next to implementation		
WHY	Contributors need design intent in the repository		
DEPENDS	1.1.8, Phase 0	DELIVERABLE	Versioned charter, ADRs, threat model, scenario spec
ACCEPTANCE	Links from README resolve and Markdown renders correctly		
1.1.10	Create and push the foundation commit		Easy / 15m
GOAL	Establish a clean baseline		
WHY	Makes subsequent behavior changes reviewable		
DEPENDS	1.1.5-1.1.9	DELIVERABLE	First commit on main
ACCEPTANCE	Local and remote main match and working tree is clean		
1.2.1	Verify supported Python installation		Easy / 10m
GOAL	Confirm the selected runtime exists		
WHY	Avoid environment drift during package initialization		
DEPENDS	1.1.10	DELIVERABLE	Recorded Python version
ACCEPTANCE	Supported interpreter launches and matches the ADR		

1.2.2	Install and verify uv	Easy / 15m
GOAL	Establish deterministic dependency management	
WHY	One package manager avoids mixed workflows	
DEPENDS	1.2.1	DELIVERABLE Working uv installation
ACCEPTANCE	uv reports a version and can resolve a trivial environment	
1.2.3	Initialize the Python package	Medium / 30m
GOAL	Create one installable package and CLI entry-point placeholder	
WHY	Gives tests and tooling a stable import path	
DEPENDS	1.2.2	DELIVERABLE pyproject.toml, lockfile, package root
ACCEPTANCE	Editable install succeeds in a clean local environment	
1.2.4	Add runtime dependencies narrowly	Easy / 20m
GOAL	Install only schemas, CLI, Kubernetes client, and selected provider SDK	
WHY	Dependency sprawl increases security and maintenance cost	
DEPENDS	1.2.3	DELIVERABLE Locked minimal runtime dependency set
ACCEPTANCE	Each dependency has a current same-day use documented in the manifest	
1.2.5	Add development quality dependencies	Easy / 20m
GOAL	Establish formatting, linting, typing, and testing	
WHY	Automated feedback is cheaper than manual cleanup	
DEPENDS	1.2.3	DELIVERABLE Locked pytest/Ruff/mypy tooling
ACCEPTANCE	Each tool starts successfully against the empty/minimal package	
1.2.6	Configure formatting and linting	Easy / 25m
GOAL	Make code style deterministic	
WHY	Codex and human edits should converge on one style	
DEPENDS	1.2.5	DELIVERABLE Tool configuration in pyproject.toml
ACCEPTANCE	Formatting check and lint check pass	
1.2.7	Configure type checking	Medium / 25m
GOAL	Define the initial strictness boundary	
WHY	Tool and state contracts benefit from static checks	
DEPENDS	1.2.5	DELIVERABLE mypy configuration
ACCEPTANCE	Type check passes; exclusions are explicit and justified	
1.2.8	Configure pytest markers	Easy / 20m
GOAL	Separate unit, integration, live-cluster, and live-model tests	
WHY	Paid/external tests must not run accidentally	
DEPENDS	1.2.5	DELIVERABLE pytest configuration and marker policy
ACCEPTANCE	Unknown markers fail and default test run excludes live dependencies	

1.2.9	Add environment-variable template	Easy / 15m
GOAL	Document required configuration without storing secrets	
WHY	Provider credentials must remain outside Git and prompts	
DEPENDS	1.2.4	DELIVERABLE <code>.env.example</code> and configuration notes
ACCEPTANCE	Template contains names only, no real credential or kubeconfig content	
1.2.10	Add VS Code workspace recommendations	Easy / 20m
GOAL	Make local commands and interpreter selection predictable	
WHY	Reduces setup variance without requiring an IDE	
DEPENDS	1.2.5-1.2.8	DELIVERABLE <code>.vscode</code> recommendations/settings
ACCEPTANCE	Settings reference project-local tools and do not contain user-specific paths	
1.3.1	Install and verify Docker	Medium / 30-90m
GOAL	Provide the runtime required by kind	
WHY	The first live scenario needs disposable containers	
DEPENDS	1.1.10	DELIVERABLE Working Docker engine
ACCEPTANCE	A basic container runs and is removed successfully	
1.3.2	Install and verify <code>kubect1</code>	Easy / 20m
GOAL	Provide an independent way to inspect and debug the lab	
WHY	Human verification must not depend on the agent	
DEPENDS	1.3.1	DELIVERABLE Working <code>kubect1</code>
ACCEPTANCE	Client launches; no production context is selected accidentally	
1.3.3	Install and verify kind	Easy / 20m
GOAL	Provide a reproducible local Kubernetes cluster	
WHY	Avoid dependence on a real or shared environment	
DEPENDS	1.3.1-1.3.2	DELIVERABLE Working kind binary
ACCEPTANCE	kind reports a version and can access Docker	
1.3.4	Audit current kubeconfig contexts	Medium / 20m
GOAL	Prevent accidental access to a real cluster	
WHY	Safety begins before application code	
DEPENDS	1.3.2	DELIVERABLE Recorded allowed local context
ACCEPTANCE	The intended kind context is identifiable and no automation defaults to another context	
1.3.5	Add a single developer command reference	Easy / 30m
GOAL	Put canonical setup/check/test commands in one place	
WHY	Reduces README drift and “which command?” ambiguity	
DEPENDS	1.2.3-1.3.4	DELIVERABLE <code>docs/contributing/local-development.md</code>
ACCEPTANCE	A clean-shell setup path is documented in execution order	

1.3.6	Commit and push the toolchain foundation		Easy / 15m
GOAL	Preserve a reproducible pre-feature baseline		
WHY	Separates setup changes from product behavior		
DEPENDS	1.2.1-1.3.5	DELIVERABLE	chore commit on main
ACCEPTANCE	Fresh clone setup succeeds and the remote is current		

1.3.7	Add starter deterministic CI		Medium / 60m
GOAL	Run format, lint, type, unit, and secret checks on pushes/PRs		
WHY	The first behavior branch should receive automated feedback		
DEPENDS	1.2.5-1.2.9, 1.3.6	DELIVERABLE	Basic GitHub Actions workflow
ACCEPTANCE	Fresh PR runs without cluster/provider secrets and required checks reflect local commands		

Phase 1 risks

Blocker or mistake	Typical beginner failure	Prevention
Tool installation consumes the day	Debugging Docker/WSL/kubeconfig without a time box	Verify prerequisites first; if Docker is blocked, complete the fixture-backed path and schedule the live cluster immediately next
Secrets enter Git	Committing .env, kubeconfig, or provider keys	Ignore them before creating them; inspect staged files before every commit
The repository starts with every future directory	Creating api, web, plugins, aws, and policies as empty placeholders	Create only directories used in the current week
Dependency choices multiply	Mixing pip, Poetry, uv, requirements files, and ad hoc installs	Use the package-manager ADR and one lockfile

Phase 1 Definition of Done

- A fresh clone can create the environment and run formatting, linting, typing, and deterministic tests.
- Docker, kubectl, and kind are available or a clearly logged environment blocker exists.
- The repository contains no secret and no unused future architecture.
- main is pushed, clean, and protected by a starter deterministic CI signal.

Phase 2 - Same-day live walking skeleton

Purpose: produce the smallest genuine AI/Kubernetes investigation today. **Estimated phase effort:** 7-11 hours after prerequisites. **Milestone:** M0.5 - first live investigation.

Learning prerequisites

Know: Deployment selectors, Pod conditions, readiness probes, events, container logs, kubeconfig contexts, structured model output. Safely defer: rollout history, Prometheus, iterative agent planning, persistence, generalized tool registries, advanced prompt optimization.

Tasks

2.1.1	Create the Day-0 feature branch	Easy / 5m
GOAL	Isolate the first behavior slice	
WHY	Keeps <code>main</code> recoverable and the review bounded	
DEPENDS	Phase 1	DELIVERABLE feat/day0-broken-readiness
ACCEPTANCE	Branch starts from current <code>main</code> and working tree is clean	

2.1.2	Define the exact CLI contract	Easy / 20m
GOAL	Freeze command inputs, exit behavior, and report modes	
WHY	Prevent implementation from inventing interface details	
DEPENDS	0.1.4-0.1.5, 2.1.1	DELIVERABLE CLI contract in quick-start draft
ACCEPTANCE	Namespace and Deployment are required; cluster context and time window are explicit; no chat input exists	

2.1.3	Define the minimal module boundaries	Medium / 20m
GOAL	Separate config, schemas, Kubernetes reads, provider call, investigation flow, and rendering	
WHY	Enables testing without premature platform abstractions	
DEPENDS	0.2.5, 2.1.2	DELIVERABLE Short module map
ACCEPTANCE	Each module has one responsibility and every module is used by the slice	

2.2.1	Define the kind cluster configuration	Easy / 20m
GOAL	Make the local environment reproducible	
WHY	The scenario must be disposable and repeatable	
DEPENDS	1.3.3-1.3.4	DELIVERABLE Versioned kind configuration
ACCEPTANCE	Cluster name/context are explicit and cannot be confused with production	

2.2.2	Create the local cluster	Medium / 15-30m
GOAL	Start an isolated investigation target	
WHY	Real Kubernetes evidence is required for credibility	
DEPENDS	2.2.1	DELIVERABLE Running kind cluster
ACCEPTANCE	Nodes are Ready and current context is the intended kind context	

2.2.3	Create the demo namespace and least-privilege identity	Medium / 45m
GOAL	Isolate the scenario and enforce read-only access	
WHY	The application, not the model, must enforce safety	
DEPENDS	2.2.2, 0.2.6	DELIVERABLE Namespace, ServiceAccount, Role, RoleBinding
ACCEPTANCE	Authorization checks allow required reads and deny create/update/patch/delete	
2.2.4	Define the healthy demo workload	Medium / 30m
GOAL	Establish a known baseline before fault injection	
WHY	Ground truth needs a controlled before/after state	
DEPENDS	2.2.3	DELIVERABLE Minimal Deployment and Service manifests
ACCEPTANCE	Deployment becomes Available before the fault is introduced	
2.2.5	Define the broken-readiness variant	Easy / 20m
GOAL	Inject exactly one known fault	
WHY	Multiple simultaneous faults would make diagnosis ambiguous	
DEPENDS	2.2.4, 0.2.8	DELIVERABLE Fault manifest or patch
ACCEPTANCE	Pods run but never become Ready for the documented probe reason	
2.2.6	Add deterministic setup, fault, and reset entry points	Medium / 40m
GOAL	Recreate the scenario without manual edits	
WHY	Repeatability is required for debugging and later evaluation	
DEPENDS	2.2.4-2.2.5	DELIVERABLE Scenario lifecycle commands/scripts
ACCEPTANCE	Setup, inject, verify, reset, and teardown each have one documented entry point	
2.2.7	Verify and record the ground truth manually	Medium / 25m
GOAL	Confirm Kubernetes displays the expected symptoms	
WHY	The agent cannot be judged against an unverified lab	
DEPENDS	2.2.6	DELIVERABLE Ground-truth observation checklist
ACCEPTANCE	Pod conditions/events expose the probe failure and no unrelated error dominates	
2.3.1	Implement and test configuration loading	Medium / 35m
GOAL	Load provider settings and kube context without exposing secrets	
WHY	Configuration is an explicit runtime boundary	
DEPENDS	1.2.9, 2.1.3	DELIVERABLE Validated settings component
ACCEPTANCE	Missing/invalid values produce clear errors and secret values are never logged	

2.3.2	Implement EvidenceRecord v0	Medium / 35m
GOAL	Give every observation stable identity and provenance	
WHY	Evidence citation is the core trust mechanism	
DEPENDS	0.2.2, 2.1.3	DELIVERABLE Schema plus unit tests
ACCEPTANCE	Valid records serialize deterministically and invalid records fail validation	
2.3.3	Implement ToolResult v0	Medium / 35m
GOAL	Normalize collector results and errors	
WHY	The flow should not interpret raw SDK exceptions	
DEPENDS	0.2.3, 2.3.2	DELIVERABLE Schema plus unit tests
ACCEPTANCE	Success, partial, timeout, forbidden, not-found, and unexpected errors are representable	
2.3.4	Implement FinalDiagnosis v0	Medium / 35m
GOAL	Validate model output before rendering	
WHY	Invalid model output must not reach the user as truth	
DEPENDS	0.2.4, 2.3.2	DELIVERABLE Schema plus unit tests
ACCEPTANCE	Required fields and confidence bounds are enforced	
2.3.5	Implement Kubernetes client construction	Hard / 45m
GOAL	Create a scoped client from an explicit context	
WHY	Avoid shell access and implicit production context	
DEPENDS	2.2.3, 2.3.1	DELIVERABLE Kubernetes client adapter
ACCEPTANCE	It uses the selected context/identity, supports dependency injection, and surfaces auth errors clearly	
2.3.6	Implement Deployment and Pod inspection	Hard / 60m
GOAL	Collect workload status, desired/available replicas, Pod phase/conditions, restarts, and container state	
WHY	These are the cheapest high-signal observations	
DEPENDS	2.3.2-2.3.5	DELIVERABLE First Kubernetes collector
ACCEPTANCE	Fixture test and live run return normalized evidence without dumping full objects	
2.3.7	Implement recent-event collection	Hard / 50m
GOAL	Collect relevant namespace/workload/Pod events within a bounded window	
WHY	Readiness failures are often visible in events	
DEPENDS	2.3.2-2.3.5	DELIVERABLE Event collector
ACCEPTANCE	Results are target-filtered, time-bounded, deduplicated, ordered, and size-limited	
2.3.8	Implement recent-log collection	Hard / 50m
GOAL	Collect bounded recent logs from relevant containers	
WHY	Logs may confirm or contradict the probe hypothesis	
DEPENDS	2.3.2-2.3.5	DELIVERABLE Log collector
ACCEPTANCE	Container selection, time/line limits, truncation, and errors are explicit	

2.3.9	Add output sanitization	Hard / 45m
GOAL	Redact obvious credentials/tokens and neutralize untrusted text before model context	
WHY	Kubernetes logs and events are untrusted inputs	
DEPENDS	0.2.7, 2.3.6-2.3.8	DELIVERABLE Sanitizer plus adversarial unit cases
ACCEPTANCE	Known secret patterns are removed and log text cannot alter runtime policy fields	
2.3.10	Build the deterministic initial-context collector	Medium / 40m
GOAL	Run the three reads in a fixed order and return an evidence set	
WHY	Same-day scope does not need an open-ended agent loop	
DEPENDS	2.3.6-2.3.9	DELIVERABLE Context collection function
ACCEPTANCE	Given fakes, it returns a stable ordered evidence package and explicit partial failures	
2.4.1	Implement the thin model-provider adapter	Hard / 45m
GOAL	Make one structured request without embedding provider details throughout the flow	
WHY	Keeps the first provider replaceable without building routing	
DEPENDS	2.3.1, 2.3.4	DELIVERABLE Single provider boundary
ACCEPTANCE	A fake provider and live provider satisfy the same narrow call contract	
2.4.2	Build the model context package	Hard / 45m
GOAL	Include objective, normalized evidence, constraints, and output schema only	
WHY	Context quality matters more than a giant system prompt	
DEPENDS	2.3.10, 2.4.1	DELIVERABLE Inspectable context builder
ACCEPTANCE	Snapshot shows no secrets/raw Kubernetes dumps and every cited observation has an evidence ID	
2.4.3	Define the bounded diagnosis instruction	Medium / 30m
GOAL	Ask for one diagnosis, uncertainty, evidence citations, and non-executed recommendation	
WHY	Prevent free-form chat and unsupported claims	
DEPENDS	0.1.5, 2.4.2	DELIVERABLE Versioned prompt/instruction asset
ACCEPTANCE	It separates facts/inferences, forbids fabricated evidence, and does not request actions	
2.4.4	Execute and validate one structured diagnosis	Hard / 60m
GOAL	Turn the collected package into a schema-valid model decision	
WHY	This is the minimum genuine AI participation	
DEPENDS	2.3.4, 2.4.1-2.4.3	DELIVERABLE Model decision step
ACCEPTANCE	Malformed output fails closed and valid output becomes a <code>FinalDiagnosis</code>	

2.4.5	Validate evidence references independently	Medium / 30m
GOAL	Reject any diagnosis citing unknown evidence IDs	
WHY	The model cannot self-authorize its own evidence	
DEPENDS	2.3.10, 2.4.4	DELIVERABLE Evidence-reference validator
ACCEPTANCE	Tests cover valid IDs, unknown IDs, duplicates, and empty evidence	

2.4.6	Render the human-readable report	Medium / 40m
GOAL	Present symptoms, evidence, conclusion, uncertainty, and recommendation clearly	
WHY	A schema alone is not a usable CLI result	
DEPENDS	2.4.4-2.4.5	DELIVERABLE Terminal report renderer plus optional JSON output
ACCEPTANCE	Human and JSON outputs represent the same validated diagnosis and no claim loses its evidence link	

2.4.7	Wire the CLI end to end	Hard / 60m
GOAL	Connect intake, collection, provider, validation, report, and exit codes	
WHY	Completes the walking skeleton	
DEPENDS	2.1.2, 2.3.10, 2.4.1-2.4.6	DELIVERABLE Runnable <code>investigate</code> command
ACCEPTANCE	Valid run succeeds; config, auth, target, tool, provider, and validation failures have distinct nonzero outcomes	

2.5.1	Add fixture-backed collector tests	Medium / 60m
GOAL	Verify normalization without a cluster	
WHY	Deterministic tests must run on every change	
DEPENDS	2.3.6-2.3.10	DELIVERABLE Unit fixtures and tests
ACCEPTANCE	Default test suite exercises healthy, broken-readiness, no-logs, forbidden, and truncated-output cases	

2.5.2	Add a fake-provider end-to-end test	Hard / 45m
GOAL	Verify the full flow without cost or network	
WHY	CI cannot depend on a paid model	
DEPENDS	2.4.1-2.4.7, 2.5.1	DELIVERABLE Deterministic E2E test
ACCEPTANCE	One command path produces the expected validated report from fixtures	

2.5.3	Run the live-cluster/no-model check	Medium / 20m
GOAL	Verify real evidence collection independently	
WHY	Separates Kubernetes bugs from model bugs	
DEPENDS	2.2.7, 2.4.7	DELIVERABLE Captured collector result
ACCEPTANCE	Collected evidence matches the manual ground truth	

2.5.4	Run the full live investigation	Hard / 20-60m
GOAL	Prove the same path works with the real cluster and provider	
WHY	This is today's core outcome	
DEPENDS	2.5.2-2.5.3	DELIVERABLE Successful live report
ACCEPTANCE	Report identifies broken readiness, cites collected evidence, expresses bounded confidence, and proposes no executed action	
2.5.5	Save a sanitized investigation trace	Medium / 30m
GOAL	Preserve input, versions, tool summaries, evidence, model output, timing, and final result	
WHY	Reproducibility begins on the first run	
DEPENDS	2.5.4	DELIVERABLE examples/traces/broken-readiness/ artifact
ACCEPTANCE	Trace contains no key, token, kubeconfig, or sensitive raw log and can be inspected offline	
2.5.6	Complete the quick start	Medium / 45m
GOAL	Let another developer recreate the live run	
WHY	A working demo that only its author can run is weak evidence	
DEPENDS	2.2.6, 2.5.4-2.5.5	DELIVERABLE docs/user/quick-start.md
ACCEPTANCE	Steps cover prerequisites, setup, fault, investigation, expected output, reset, and cleanup	
2.5.7	Run the Day-0 quality gate	Medium / 30m
GOAL	Execute format, lint, type, unit, fake-E2E, and live smoke checks	
WHY	Prevent "works once" from becoming the baseline	
DEPENDS	2.5.1-2.5.6	DELIVERABLE Green quality checklist
ACCEPTANCE	All deterministic checks pass and live result is linked from the checklist	
2.5.8	Review for prohibited scope and permissions	Hard / 30m
GOAL	Remove accidental abstractions, writes, shell execution, and secret exposure	
WHY	Same-day speed cannot erase core guardrails	
DEPENDS	2.5.7	DELIVERABLE Reviewed diff and security checklist
ACCEPTANCE	No write-capable Kubernetes method, arbitrary command path, unused future module, or committed secret exists	
2.5.9	Merge and tag the walking skeleton	Easy / 20m
GOAL	Establish a stable checkpoint	
WHY	The project now has a recoverable first working version	
DEPENDS	2.5.8	DELIVERABLE Merged PR and optional v0.0.1 tag
ACCEPTANCE	main is green, remote is current, and README accurately states the narrow capability	

2.5.10	Record observed follow-up defects		Easy / 20m
GOAL	Convert real-run findings into bounded backlog items		
WHY	Do not repair unrelated issues inside the completed slice		
DEPENDS	2.5.4-2.5.9	DELIVERABLE	Prioritized issue list
ACCEPTANCE	Each issue has reproduction, impact, scope, and target phase		

Phase 2 risks

Blocker or mistake	Typical beginner failure	Prevention
The model is asked to operate Kubernetes	Letting it generate shell commands or choose arbitrary SDK calls	Deterministic collectors run first; the model only diagnoses normalized evidence
One lucky output is treated as success	Rerunning until the model says the expected phrase	Use schema validation, evidence-ID checks, fake E2E, and record every live attempt
Raw objects overflow context	Sending complete Pod/Deployment objects and thousands of log lines	Normalize, filter, cap, and mark truncation before the provider call
The scenario is accidentally ambiguous	Demo image, networking, and probe all fail at once	Prove the healthy baseline, inject one fault, and manually verify ground truth
Today becomes a platform build	Adding registries, DB, worker, API, generic planner, or plugins	Keep the deterministic single-step flow; Phase 4 owns generalization
Sleep-deprived work produces unsafe shortcuts	Pushing secrets or accepting code not understood	If working after a night shift, stop after repository/scenario setup and resume the live model path after sleep

Phase 2 Definition of Done

- The live kind scenario is reproducible, read-only, and has one verified root cause.
- The CLI collects real bounded evidence through the Kubernetes client.
- The model returns a schema-valid diagnosis whose evidence IDs are independently checked.
- Deterministic tests cover the entire flow without a cluster or model call.
- A sanitized trace and reproducible quick start exist.
- The result is merged to green main; no future platform layer was added.

Phase 3 - Complete the reliable read-only Kubernetes vertical slice

Purpose: turn the walking skeleton into a repeatable investigator for three known Kubernetes failures.

Estimated phase effort: 35-55 hours. **Milestone/release:** M1 / v0.1.0 - read-only Kubernetes investigation.

Learning prerequisites

Know: rollout mechanics, events, container states, probes, Secrets/configuration references, Prometheus fundamentals, retries versus non-retryable errors. Safely defer: generalized plugin APIs, persistent service storage, write actions, arbitrary PromQL generation, cloud metrics.

Tasks

3.1.1	Freeze the Day-0 regression baseline	Medium / 60m
GOAL	Preserve the known working behavior before expansion	
WHY	Refactoring without a baseline can silently break the only useful path	
DEPENDS	Phase 2	DELIVERABLE Versioned fixtures, expected report assertions, live trace
ACCEPTANCE	Default deterministic suite proves the broken-readiness path still works	
3.1.2	Formalize collector input contracts	Medium / 60m
GOAL	Give all Kubernetes collectors the same target/time/output-limit inputs	
WHY	Predictable tools reduce model and orchestration ambiguity	
DEPENDS	3.1.1	DELIVERABLE Typed collector request schemas
ACCEPTANCE	Invalid cluster, namespace, kind, name, time, and limits fail before SDK calls	
3.1.3	Add rollout-history collection	Hard / 2h
GOAL	Correlate health changes with Deployment revisions	
WHY	The flagship use case begins after a deployment	
DEPENDS	3.1.2	DELIVERABLE Normalized rollout evidence collector
ACCEPTANCE	It returns revision/image/change timing where available and represents missing history explicitly	
3.1.4	Improve multi-container log selection	Hard / 2h
GOAL	Handle init, sidecar, current, and previous container logs safely	
WHY	Real Pods are not always single-container happy paths	
DEPENDS	3.1.2	DELIVERABLE Bounded multi-container log behavior
ACCEPTANCE	Selection is deterministic, previous crash logs are supported, and total output cap is enforced	

3.1.5	Correlate and deduplicate events	Hard / 2h
GOAL	Group repeated events and connect them to workload/Pod owners	
WHY	Raw event streams are noisy and can dominate context	
DEPENDS	3.1.2	DELIVERABLE Correlated event summary
ACCEPTANCE	Repeated events retain count/first/last timestamps and irrelevant objects are excluded	
3.1.6	Add Kubernetes-object summarization	Hard / 2h
GOAL	Preserve diagnostic fields while removing metadata noise	
WHY	Full objects waste context and may contain sensitive data	
DEPENDS	3.1.2	DELIVERABLE Tested Deployment/ReplicaSet/Pod summaries
ACCEPTANCE	Golden tests prove required failure fields survive and managedFields/annotations noise is excluded	
3.1.7	Define the initial metric questions	Medium / 60m
GOAL	Select a fixed small set of workload-health questions	
WHY	The model should not receive unrestricted PromQL initially	
DEPENDS	0.1.3, 3.1.1	DELIVERABLE Metric question catalog
ACCEPTANCE	Each question names required labels, window, interpretation, and unavailable-data behavior	
3.1.8	Add local Prometheus to the lab	Hard / 3h
GOAL	Produce queryable workload/cluster metrics	
WHY	Metrics are one planned initial evidence source	
DEPENDS	3.1.7, 2.2.6	DELIVERABLE Reproducible local Prometheus configuration
ACCEPTANCE	Prometheus starts with the scenario and returns data for every supported question or a documented limitation	
3.1.9	Implement the typed Prometheus query tool	Hard / 3h
GOAL	Turn catalog questions into normalized evidence	
WHY	Metrics need the same safety/provenance contract as Kubernetes reads	
DEPENDS	3.1.7-3.1.8, 2.3.3	DELIVERABLE Prometheus collector with tests
ACCEPTANCE	Inputs are allow-listed, time-bounded, output-capped, and failures are normalized	
3.1.10	Add metric summarization	Hard / 2h
GOAL	Reduce series into relevant windows, changes, and anomalies	
WHY	Raw series are unsuitable model context	
DEPENDS	3.1.9	DELIVERABLE Metric evidence summarizer
ACCEPTANCE	Tests preserve values/timestamps/labels required for conclusions and mark missing/stale data	

3.2.1	Define NextStepDecision v0	Hard / 90m
GOAL	Structure the model's short-horizon choice	
WHY	The mature flow needs controlled decisions, not one giant diagnosis prompt	
DEPENDS	3.1.1	DELIVERABLE Decision schema and invariants
ACCEPTANCE	It supports select-tool, diagnose, and escalate/stop outcomes with rationale and expected evidence	
3.2.2	Define Hypothesis v0	Hard / 90m
GOAL	Track possible causes and what would confirm/reject them	
WHY	A reliable investigator must manage uncertainty explicitly	
DEPENDS	3.2.1, 0.2.2	DELIVERABLE Hypothesis schema
ACCEPTANCE	Supporting/contradicting/missing evidence, confidence, status, and best next check are required	
3.2.3	Expand InvestigationState v1	Hard / 2h
GOAL	Store hypotheses, unknowns, tool history, budgets, and current phase	
WHY	State must replace hidden conversational memory	
DEPENDS	3.2.1-3.2.2	DELIVERABLE Versioned state schema and migration note
ACCEPTANCE	State round-trips deterministically and rejects invalid phase/status combinations	
3.2.4	Define deterministic initial collection	Medium / 60m
GOAL	Gather cheap Deployment/Pod/event/rollout evidence before planning	
WHY	Avoid spending a model call to discover obvious symptoms	
DEPENDS	3.1.3-3.1.6, 3.2.3	DELIVERABLE Initial-collection policy
ACCEPTANCE	The same bounded baseline runs for every supported scenario	
3.2.5	Implement the short-horizon decision step	Hard / 4h
GOAL	Let the model choose the next one tool or stop	
WHY	This introduces real multi-step reasoning while remaining bounded	
DEPENDS	3.1.2-3.1.10, 3.2.1-3.2.4	DELIVERABLE Schema-validated decision call
ACCEPTANCE	Only registered read tools can be selected and invalid choices fail closed	
3.2.6	Implement evidence/state update	Hard / 3h
GOAL	Merge new evidence and update hypotheses after each decision	
WHY	Tool results must change explicit state, not only prompt text	
DEPENDS	3.2.2-3.2.5	DELIVERABLE Deterministic state reducer
ACCEPTANCE	Replayed identical events produce identical state and preserve provenance	
3.2.7	Add execution budgets	Hard / 2h
GOAL	Bound calls, time, output volume, tokens, and estimated cost	
WHY	Bounded autonomy is a core principle	
DEPENDS	3.2.3, 3.2.5	DELIVERABLE Configurable run budget
ACCEPTANCE	Every dimension is checked before work and terminal budget exhaustion is reported honestly	

3.2.8	Add duplicate-call detection	Hard / 2h
GOAL	Stop repeated equivalent tool requests	
WHY	Agent loops waste money and hide weak planning	
DEPENDS	3.2.6-3.2.7	DELIVERABLE Canonical call fingerprint and policy
ACCEPTANCE	Exact and semantically equivalent repeated calls are detected in tests	
3.2.9	Add explicit termination rules	Hard / 2h
GOAL	End on sufficient evidence, bounded uncertainty, budget exhaustion, or hard failure	
WHY	The model must not control termination alone	
DEPENDS	3.2.5-3.2.8	DELIVERABLE Termination evaluator
ACCEPTANCE	Every run reaches a named terminal reason within configured limits	
3.2.10	Generate the final report from final state	Hard / 2h
GOAL	Separate investigation decisions from report prose	
WHY	The final report must reflect validated state only	
DEPENDS	3.2.6, 3.2.9	DELIVERABLE State-to-diagnosis/report path
ACCEPTANCE	Unsupported claims/evidence fail validation and terminal reason appears in output	
3.3.1	Specify the missing-Secret scenario	Medium / 60m
GOAL	Add a distinct configuration failure with ground truth	
WHY	Two scenarios expose hardcoded readiness logic	
DEPENDS	3.2.10	DELIVERABLE Scenario specification
ACCEPTANCE	Symptoms, relevant/distracting evidence, expected diagnosis, and forbidden actions are defined	
3.3.2	Implement and verify the missing-Secret fault	Hard / 2h
GOAL	Make the scenario reproducible in the local lab	
WHY	The agent must investigate live state, not a fixture alone	
DEPENDS	3.3.1, 2.2.6	DELIVERABLE Setup/fault/reset/verify lifecycle
ACCEPTANCE	Repeated resets return to healthy and fault injection yields the documented error	
3.3.3	Add deterministic missing-Secret tests	Medium / 90m
GOAL	Prove collectors and reasoning input support the scenario	
WHY	Prevent model reruns from masking data bugs	
DEPENDS	3.3.2	DELIVERABLE Fixtures and assertions
ACCEPTANCE	Default suite captures the necessary evidence without encoding the final phrase in production logic	

3.3.4	Specify the image-pull-failure scenario	Medium / 60m
GOAL	Add a distinct supply/deployment failure	
WHY	It exercises different events and container states	
DEPENDS	3.2.10	DELIVERABLE Scenario specification
ACCEPTANCE	Ground truth and acceptable uncertainty are explicit	
3.3.5	Implement and verify the image-pull fault	Hard / 2h
GOAL	Reproduce <code>ErrImagePull</code> /backoff behavior	
WHY	Third live scenario is the M1 reliability gate	
DEPENDS	3.3.4, 2.2.6	DELIVERABLE Setup/fault/reset/verify lifecycle
ACCEPTANCE	The fault is stable enough for repeated investigation and cleanup is deterministic	
3.3.6	Add deterministic image-pull tests	Medium / 90m
GOAL	Protect normalization and context behavior	
WHY	Event/backoff output varies and needs normalization	
DEPENDS	3.3.5	DELIVERABLE Fixtures and assertions
ACCEPTANCE	Fixture variants produce semantically consistent evidence	
3.4.1	Centralize timeout behavior	Hard / 2h
GOAL	Apply per-tool and run-level deadlines	
WHY	External systems must not hang an investigation	
DEPENDS	3.1.2-3.1.10, 3.2.7	DELIVERABLE Timeout policy and tests
ACCEPTANCE	Slow fake tools terminate, record timing, and do not exceed run budget materially	
3.4.2	Centralize retry behavior	Hard / 2h
GOAL	Retry only transient, idempotent reads with capped backoff	
WHY	Blind retries amplify load and delay diagnosis	
DEPENDS	3.4.1	DELIVERABLE Retry classifier/policy
ACCEPTANCE	Forbidden, invalid, and not-found failures are not retried; transient cases respect caps	
3.4.3	Enforce total context/output limits	Hard / 2h
GOAL	Cap evidence count and bytes across all tools	
WHY	Per-tool caps alone do not protect the model context	
DEPENDS	3.2.6-3.2.7	DELIVERABLE Context budget enforcement
ACCEPTANCE	Oversized tests truncate by documented priority without corrupting provenance	
3.4.4	Expand redaction adversarial tests	Hard / 2h
GOAL	Cover secrets in logs, events, environment references, and malicious instructions	
WHY	Operational data is untrusted	
DEPENDS	2.3.9, 3.1.6	DELIVERABLE Security regression suite
ACCEPTANCE	Known credential formats and prompt-injection strings cannot alter constraints or appear in saved context	

3.4.5	Make traces replayable		Hard / 3h
GOAL	Re-run the state reducer and report validation from a saved trace		
WHY	Reproducibility is required for debugging/evaluation		
DEPENDS	3.2.6, 3.2.10	DELIVERABLE	Trace format plus replay command
ACCEPTANCE	A saved run replays offline to the same final structured state		
3.4.6	Run a repeatability matrix		Hard / 4-6h
GOAL	Execute each scenario repeatedly under fixed configuration		
WHY	M1 requires repeatability, not three screenshots		
DEPENDS	3.3.1-3.4.5	DELIVERABLE	Run matrix with outcomes/cost/latency
ACCEPTANCE	At least five consecutive runs per scenario meet documented diagnosis/evidence thresholds without prompt editing		
3.4.7	Document observed failure modes		Easy / 60m
GOAL	Tell users where the agent is uncertain or unsupported		
WHY	Honest limitations are a credibility signal		
DEPENDS	3.4.6	DELIVERABLE	docs/user/limitations.md
ACCEPTANCE	All failures from the matrix are categorized and no limitation is hidden		
3.4.8	Upgrade pull-request CI		Hard / 3h
GOAL	Add fake-E2E, schema compatibility, dependency, and full deterministic checks		
WHY	M1 needs stronger protection than the starter workflow		
DEPENDS	1.3.7, 3.4.5	DELIVERABLE	Expanded GitHub Actions workflow
ACCEPTANCE	Fresh PR receives required green/red checks; no live key or cluster is required		
3.4.9	Protect <code>main</code>		Easy / 20m
GOAL	Require CI and reviewed PR flow		
WHY	Prevent accidental direct breakage		
DEPENDS	3.4.8	DELIVERABLE	Branch protection settings
ACCEPTANCE	Direct push is restricted as intended and required checks are enforced		
3.4.10	Publish <code>v0.1.0</code>		Medium / 2h
GOAL	Release the first serious user-facing capability		
WHY	Milestones should produce demonstrable artifacts		
DEPENDS	3.4.6-3.4.9	DELIVERABLE	Tagged release, changelog, quick start, known limitations, sample trace
ACCEPTANCE	A clean-machine test follows docs and completes one supported investigation		

Phase 3 risks

Blocker or mistake	Typical beginner failure	Prevention
The “agent loop” becomes an unrestricted loop	Letting the model call any tool indefinitely	Enforce typed choices, allow-list, budgets, duplicate detection, and deterministic termination
Scenario-specific prompt patches accumulate	Adding special instructions after every failed run	Fix missing evidence/contracts first; measure changes across all scenarios
Prometheus becomes a dashboard project	Building broad telemetry before a diagnostic question exists	Implement only the fixed metric question catalog
Success is judged manually	Picking favorable live outputs	Use five-run matrices, trace every attempt, and document failures

Phase 3 Definition of Done

- Three live scenarios are diagnosed repeatedly without manual prompt editing.
- Initial collection plus bounded short-horizon planning uses only typed read tools.
- Budgets, retries, timeouts, duplicate detection, redaction, and termination are enforced by software.
- Traces replay offline.
- Required deterministic CI protects `main`.
- `v0.1.0` has reproducible docs, sample traces, and honest limitations.

Phase 4 - Reusable agent runtime and tool system

Purpose: extract the proven slice into a model-independent, reusable investigation runtime. **Estimated phase effort:** 45-70 hours. **Milestone/release:** M2 / v0.2.0 - reusable runtime.

Learning prerequisites

Know: state machines, dependency inversion, schema evolution, idempotency, persistence transactions, provider/tool error taxonomies. Safely defer: external plugins, distributed workers, provider routing, vector retrieval, write tools.

Tasks

4.1.1	Inventory proven interfaces	Medium / 2h
GOAL	Identify which Phase 3 boundaries have real usage	
WHY	Generalize evidence, not imagined future needs	
DEPENDS	Phase 3	DELIVERABLE Interface inventory with pain points
ACCEPTANCE	Every proposed runtime abstraction cites at least two current call sites or a tested cross-cutting requirement	

4.1.2	Define the investigation state machine	Hard / 3h
GOAL	Name states, events, transitions, guards, and terminal outcomes	
WHY	Explicit control is safer and replayable	
DEPENDS	4.1.1	DELIVERABLE Versioned state-machine specification
ACCEPTANCE	No transition depends on hidden chat text and invalid transitions are enumerable	

4.1.3	Implement state transition validation	Hard / 4h
GOAL	Enforce the machine independently of the model	
WHY	The model proposes; software controls execution	
DEPENDS	4.1.2	DELIVERABLE Transition engine with invariant tests
ACCEPTANCE	Invalid transitions fail without partial state mutation	

4.1.4	Separate commands, events, and state reduction	Hard / 4h
GOAL	Make runtime changes deterministic and replayable	
WHY	Clean event flow simplifies persistence/evaluation	
DEPENDS	4.1.3	DELIVERABLE Runtime event model and reducer
ACCEPTANCE	Replaying an event sequence reconstructs identical state	

4.1.5	Add cancellation and terminal failure semantics	Hard / 2h
GOAL	Stop safely on user cancellation or unrecoverable errors	
WHY	Every lifecycle needs explicit exits	
DEPENDS	4.1.2-4.1.4	DELIVERABLE Cancellation/failure events and tests
ACCEPTANCE	Cancellation prevents new calls, records reason, and preserves prior trace	

4.2.1	Define ToolDefinition v1	Hard / 3h
GOAL	Declare name, purpose, input/output schemas, permissions, mutation, idempotency, timeout, sensitivity, and failures	
WHY	Typed contracts are a central project asset	
DEPENDS	4.1.1	DELIVERABLE Tool-definition schema
ACCEPTANCE	All required metadata validates and incompatible definitions fail registration	

4.2.2	Implement the tool registry	Hard / 3h
GOAL	Register, enumerate, resolve, and reject duplicate tools	
WHY	The orchestrator must not hardcode integrations	
DEPENDS	4.2.1	DELIVERABLE Registry with tests
ACCEPTANCE	Lookup is deterministic and duplicate/incompatible names fail before runtime	

4.2.3	Implement input/output schema validation	Hard / 3h
GOAL	Validate both model request and adapter result	
WHY	External tools and models are untrusted boundaries	
DEPENDS	4.2.1-4.2.2	DELIVERABLE Validation layer
ACCEPTANCE	Invalid request never reaches adapter; invalid output becomes a normalized tool failure	

4.2.4	Implement permission preflight	Hard / 3h
GOAL	Check investigation grants against tool metadata	
WHY	Read-only safety must survive new tools	
DEPENDS	4.2.1-4.2.2	DELIVERABLE Permission evaluator
ACCEPTANCE	A tool lacking an explicit grant cannot execute even if selected by the model	

4.2.5	Implement common execution middleware	Hard / 5h
GOAL	Centralize timing, retry, timeout, limits, redaction, and audit events	
WHY	Adapters should not reimplement runtime controls	
DEPENDS	3.4.1-3.4.4, 4.2.3-4.2.4	DELIVERABLE Tool execution pipeline
ACCEPTANCE	Every registered tool receives the same controls and integration tests prove ordering	

4.2.6	Define stable tool error taxonomy	Hard / 2h
GOAL	Normalize invalid input, auth, permission, not-found, unavailable, timeout, rate-limit, partial, and internal failures	
WHY	The planner needs reliable failure meaning	
DEPENDS	4.2.5	DELIVERABLE Error taxonomy and mapping guide
ACCEPTANCE	Kubernetes and Prometheus errors map to the same categories without losing adapter details	
4.2.7	Migrate Kubernetes tools to the registry	Hard / 4h
GOAL	Prove the contract against a complex adapter family	
WHY	First independent implementation of the runtime contract	
DEPENDS	4.2.1-4.2.6	DELIVERABLE Registered Kubernetes tool package
ACCEPTANCE	Phase 3 scenarios pass without orchestrator-specific Kubernetes branches	
4.2.8	Migrate Prometheus tools to the registry	Hard / 3h
GOAL	Prove a second independent package	
WHY	M2 requires more than one adapter shape	
DEPENDS	4.2.1-4.2.6	DELIVERABLE Registered Prometheus tool package
ACCEPTANCE	Same middleware and permission system execute metric tools without special cases	
4.3.1	Define ModelCapabilities and provider contract	Hard / 3h
GOAL	Describe structured output/tool support, context, streaming, cost, and model identity	
WHY	Core behavior should depend on capability, not vendor SDK	
DEPENDS	4.1.1	DELIVERABLE Provider interface specification
ACCEPTANCE	Current provider maps cleanly and unsupported capabilities fail at startup	
4.3.2	Migrate the initial provider	Hard / 3h
GOAL	Remove provider SDK knowledge from planner/runtime	
WHY	Enables later model comparison without routing complexity	
DEPENDS	4.3.1	DELIVERABLE Provider adapter
ACCEPTANCE	Phase 3 runs pass through the interface and provider-specific types do not leak into core	
4.3.3	Build a deterministic fake provider	Medium / 2h
GOAL	Script decisions and malformed/failure responses	
WHY	Runtime tests need complete control and zero cost	
DEPENDS	4.3.1	DELIVERABLE Fake provider package
ACCEPTANCE	Tests can induce valid choice, invalid schema, timeout, rate-limit, and provider failure	

4.3.4	Add model request middleware	Hard / 4h
GOAL	Record timing, token/cost estimates, retry policy, redacted request metadata, and response validation	
WHY	Model calls need the same auditability as tools	
DEPENDS	4.3.1-4.3.3	DELIVERABLE Provider execution pipeline
ACCEPTANCE	Every call produces audit events and retry never duplicates a completed decision	
4.4.1	Define trace/persistence schema v1	Hard / 3h
GOAL	Store investigation, state events, evidence, tool/model calls, and final report	
WHY	Reproducibility needs a durable system of record	
DEPENDS	4.1.4, 4.2.5, 4.3.4	DELIVERABLE Relational schema/ERD
ACCEPTANCE	Every replay-required datum has ownership, ordering, and retention classification	
4.4.2	Implement a repository interface	Hard / 3h
GOAL	Keep runtime logic independent of SQLite	
WHY	Service mode will need a different backend	
DEPENDS	4.4.1	DELIVERABLE Persistence contract
ACCEPTANCE	In-memory and SQLite implementations can satisfy the same contract tests	
4.4.3	Implement SQLite storage	Hard / 5h
GOAL	Persist local investigations atomically	
WHY	Local durability improves debugging without operating a database service	
DEPENDS	4.4.1-4.4.2	DELIVERABLE SQLite repository and migrations
ACCEPTANCE	Interrupted writes do not produce a falsely completed investigation and migrations work from empty DB	
4.4.4	Add trace inspection and replay CLI commands	Hard / 3h
GOAL	Make persisted state useful to engineers	
WHY	Audit data hidden in a DB is not a product feature	
DEPENDS	4.4.3	DELIVERABLE CLI views for list/show/replay
ACCEPTANCE	A completed and failed run can be inspected and replayed without provider/cluster access	
4.5.1	Centralize hypothesis management	Hard / 3h
GOAL	Apply consistent confidence/status/evidence rules	
WHY	Scattered hypothesis updates are hard to test	
DEPENDS	3.2.2, 4.1.4	DELIVERABLE Hypothesis service/reducer
ACCEPTANCE	State invariants prevent cited missing evidence and invalid confidence/status transitions	

4.5.2	Centralize budget accounting	Hard / 3h
GOAL	Account for tool calls, model calls, time, tokens, cost, and bytes	
WHY	Limits must survive new integrations	
DEPENDS	3.2.7, 4.1.4	DELIVERABLE Budget component
ACCEPTANCE	Replay reconstructs the same consumption and preflight blocks over-budget work	
4.5.3	Centralize progress/loop evaluation	Hard / 3h
GOAL	Detect repeated state, repeated calls, and no-information progress	
WHY	General runtime needs more than exact-call detection	
DEPENDS	3.2.8, 4.1.4	DELIVERABLE Progress evaluator
ACCEPTANCE	Synthetic loops terminate with a clear reason and do not exceed caps	
4.5.4	Add runtime contract tests	Hard / 5h
GOAL	Test orchestration with fake tools/providers/storage	
WHY	Integration-specific tests cannot prove core behavior	
DEPENDS	4.1.3-4.5.3	DELIVERABLE Comprehensive core test suite
ACCEPTANCE	Tests cover success, partial evidence, retries, provider failure, loop, cancellation, budget, and persistence failure	
4.5.5	Add schema compatibility checks	Hard / 2h
GOAL	Detect breaking changes to persisted and extension-facing schemas	
WHY	0.x still needs intentional evolution	
DEPENDS	4.2.1, 4.3.1, 4.4.1	DELIVERABLE CI schema snapshots/checks
ACCEPTANCE	Unreviewed incompatible schema change fails CI and requires a migration/changelog entry	
4.5.6	Measure the refactor against M1	Hard / 4h
GOAL	Prove reuse did not reduce investigation quality	
WHY	Architecture is only valuable if behavior survives	
DEPENDS	4.2.7-4.5.5	DELIVERABLE Before/after repeatability report
ACCEPTANCE	All M1 scenarios meet or beat baseline accuracy, tool-call, latency, and safety bounds	
4.5.7	Document tool authoring internally	Medium / 2h
GOAL	Explain the exercised contract before public plugin stabilization	
WHY	New built-in tools should follow one path	
DEPENDS	4.2.7-4.2.8	DELIVERABLE docs/contributing/tool-authoring.md
ACCEPTANCE	A maintainer can add a fake diagnostic tool using only the guide and contract tests	

4.5.8	Publish v0.2.0		Medium / 2h
GOAL	Release the reusable runtime milestone		
WHY	Creates a stable checkpoint for the incident lab		
DEPENDS	4.5.4-4.5.7	DELIVERABLE	Release, changelog, migration note, architecture update
ACCEPTANCE	Two tool packages use one runtime contract and M1 behavior remains green		

Phase 4 risks

Blocker or mistake	Typical beginner failure	Prevention
“Reusable” becomes speculative framework design	Adding extension points with one caller	Require evidence from current tools or a cross-cutting runtime requirement
Refactoring breaks the investigation	Celebrating cleaner modules while accuracy drops	Keep the M1 repeatability matrix as a release gate
Provider abstraction becomes model routing	Building fallback/routing before comparison data exists	Support one live provider and one deterministic fake only
Persistence leaks into every module	Passing DB models through core logic	Persist domain events through a narrow repository interface

Phase 4 Definition of Done

- An explicit state machine controls every investigation transition.
- Kubernetes and Prometheus packages use the same validated registry and execution middleware.
- A model-provider interface, live adapter, and deterministic fake exist.
- Local runs persist and replay through SQLite.
- Runtime contract tests cover failures, budgets, loops, cancellation, and audit behavior.
- M1 quality does not regress and v0.2.0 is released.

Phase 5 - Reproducible incident lab

Purpose: create realistic, resettable environments with known ground truth. **Estimated phase effort:** 55-85 hours. **Milestone:** M3 - reproducible incident lab.

Learning prerequisites

Know: Kubernetes manifests, health probes, resource limits, Services/DNS, telemetry basics, deterministic setup/teardown, test fixture design. Safely defer: chaos-engineering platforms, service meshes, large microservice demos, cloud clusters, polished dashboards.

Tasks

5.1.1	Define incident-lab scope	Easy / 45m
GOAL	Limit the lab to one small application and 5-7 diagnostic scenarios	
WHY	The lab supports evaluation; it is not a second product	
DEPENDS	Phase 4	DELIVERABLE Lab charter/non-goals
ACCEPTANCE	It rejects excessive services, UI work, chaos platforms, and production claims	
5.1.2	Define ScenarioDefinition v1	Hard / 3h
GOAL	Standardize metadata, prerequisites, setup, fault, verification, reset, ground truth, evidence, remediation, and forbidden actions	
WHY	Scenarios need a common machine-readable contract	
DEPENDS	5.1.1, 4.5.5	DELIVERABLE Versioned scenario schema
ACCEPTANCE	Existing three scenarios validate after migration and invalid lifecycle definitions fail	
5.1.3	Define ground-truth evidence matching	Hard / 2h
GOAL	Describe required, optional, distracting, and forbidden evidence	
WHY	Later scoring needs more than a root-cause label	
DEPENDS	5.1.2	DELIVERABLE Ground-truth evidence schema
ACCEPTANCE	Evidence expectations can be matched semantically without hardcoding runtime output IDs	
5.1.4	Define scenario lifecycle states	Hard / 2h
GOAL	Standardize absent, healthy, faulted, detectable, reset, and failed states	
WHY	Automation must know whether a scenario is truly ready	
DEPENDS	5.1.2	DELIVERABLE Lifecycle state machine
ACCEPTANCE	Illegal transitions and timeouts are explicit	
5.1.5	Add scenario CLI commands	Hard / 4h
GOAL	Expose list, describe, setup, inject, verify, reset, and teardown	
WHY	Contributors need one repeatable interface	
DEPENDS	5.1.2-5.1.4	DELIVERABLE Scenario command group
ACCEPTANCE	Commands return machine-readable status and nonzero failure outcomes	

5.2.1	Design the minimal demo application	Hard / 2h
GOAL	Choose the smallest topology that can exhibit dependency and configuration failures	
WHY	A realistic target is useful; microservice theater is not	
DEPENDS	5.1.1	DELIVERABLE Demo app design
ACCEPTANCE	At most two app services plus telemetry support all planned scenarios	
5.2.2	Implement deterministic application behavior	Hard / 5h
GOAL	Make health, dependency, memory, and configuration signals predictable	
WHY	Ground truth must not depend on random app behavior	
DEPENDS	5.2.1	DELIVERABLE Versioned demo app/container
ACCEPTANCE	Healthy mode is stable and each supported fault has an observable, documented signal	
5.2.3	Add application metrics	Hard / 3h
GOAL	Expose only signals used by planned investigations	
WHY	Prometheus evidence must be meaningful	
DEPENDS	5.2.2, 3.1.7	DELIVERABLE Metrics and tests
ACCEPTANCE	Healthy/faulted values match scenario expectations and label cardinality is bounded	
5.2.4	Add structured application logs	Hard / 3h
GOAL	Produce bounded, correlated operational evidence	
WHY	Log scenarios need stable search/summarization input	
DEPENDS	5.2.2	DELIVERABLE Structured log events
ACCEPTANCE	Events include timestamp, severity, service, event type, and correlation fields without secrets	
5.2.5	Package immutable demo images	Hard / 3h
GOAL	Pin exactly what scenarios deploy	
WHY	Mutable tags make failures irreproducible	
DEPENDS	5.2.2-5.2.4	DELIVERABLE Versioned container images/build process
ACCEPTANCE	Image digest is recorded and clean rebuild produces functionally equivalent behavior	
5.2.6	Build the base Kubernetes environment	Hard / 4h
GOAL	Deploy healthy app, metrics collection, and minimum permissions	
WHY	Every scenario needs the same verified starting point	
DEPENDS	5.2.3-5.2.5	DELIVERABLE Base manifests/overlays
ACCEPTANCE	One command reaches healthy state and readiness/metrics/log checks pass	
5.2.7	Add environment health verification	Hard / 3h
GOAL	Detect broken lab infrastructure before agent evaluation	
WHY	A bad lab must not be scored as an agent failure	
DEPENDS	5.2.6	DELIVERABLE Lab preflight checks
ACCEPTANCE	Checks distinguish app, cluster, metrics, credentials, and image failures	

5.3.1	Migrate broken-readiness to ScenarioDefinition v1	Medium / 2h
GOAL	Prove backward compatibility	
WHY	Existing scenarios are the first contract consumers	
DEPENDS	5.1.2-5.1.5, 5.2.6	DELIVERABLE Versioned scenario package
ACCEPTANCE	Full lifecycle is automated and ground truth validates	
5.3.2	Migrate missing-Secret to ScenarioDefinition v1	Medium / 2h
GOAL	Standardize the second scenario	
WHY	Exposes configuration/secret lifecycle requirements	
DEPENDS	5.3.1	DELIVERABLE Versioned scenario package
ACCEPTANCE	Setup/inject/verify/investigate/reset succeeds twice consecutively	
5.3.3	Migrate image-pull failure to ScenarioDefinition v1	Medium / 2h
GOAL	Standardize the third scenario	
WHY	Exposes external image/backoff timing behavior	
DEPENDS	5.3.1	DELIVERABLE Versioned scenario package
ACCEPTANCE	Verification tolerates bounded backoff timing without false readiness	
5.3.4	Specify memory-limit termination	Medium / 90m
GOAL	Define an OOMKilled scenario with evidence and distractions	
WHY	Adds resource/container-history reasoning	
DEPENDS	5.2.6, 5.1.3	DELIVERABLE Scenario specification
ACCEPTANCE	Ground truth names required previous-state/resource evidence and safe recommendation	
5.3.5	Implement and verify memory-limit termination	Hard / 4h
GOAL	Make OOM behavior repeatable and resettable	
WHY	Resource failures are central Kubernetes incidents	
DEPENDS	5.3.4	DELIVERABLE Scenario package
ACCEPTANCE	OOM occurs within a bounded time, previous state is observable, and reset becomes healthy	
5.3.6	Specify invalid service configuration	Medium / 90m
GOAL	Define a configuration-induced application failure	
WHY	Exercises logs plus Kubernetes configuration correlation	
DEPENDS	5.2.6, 5.1.3	DELIVERABLE Scenario specification
ACCEPTANCE	Relevant and distracting evidence are explicit	

5.3.7	Implement and verify invalid service configuration	Hard / 4h
GOAL	Produce a stable misconfiguration signal	
WHY	Adds a failure not diagnosed by Pod phase alone	
DEPENDS	5.3.6	DELIVERABLE Scenario package
ACCEPTANCE	Pod may run while app health/logs demonstrate the documented cause	
5.3.8	Specify dependency timeout	Hard / 2h
GOAL	Define a downstream latency/unavailability incident	
WHY	Exercises service/metric/log correlation and uncertainty	
DEPENDS	5.2.1, 5.1.3	DELIVERABLE Scenario specification
ACCEPTANCE	Expected diagnosis distinguishes local app health from dependency health	
5.3.9	Implement and verify dependency timeout	Hard / 5h
GOAL	Reproduce bounded downstream failure	
WHY	Adds a production-inspired incomplete-signal case	
DEPENDS	5.3.8	DELIVERABLE Scenario package
ACCEPTANCE	Fault is detectable through at least two evidence types and reset is deterministic	
5.3.10	Specify failed/partial rollout	Hard / 2h
GOAL	Define mixed old/new ReplicaSet behavior	
WHY	Directly supports the flagship post-deployment workflow	
DEPENDS	5.2.6, 5.1.3	DELIVERABLE Scenario specification
ACCEPTANCE	Ground truth covers revision, partial availability, and safe recommendation	
5.3.11	Implement and verify failed/partial rollout	Hard / 5h
GOAL	Create a repeatable mixed-revision failure	
WHY	Forces rollout-history and Pod correlation	
DEPENDS	5.3.10	DELIVERABLE Scenario package
ACCEPTANCE	Both revisions and failure symptoms are visible and cleanup restores one healthy revision	
5.4.1	Add scenario isolation	Hard / 3h
GOAL	Prevent residue from one fault contaminating another	
WHY	Cross-test leakage destroys evaluation validity	
DEPENDS	5.3.1-5.3.11	DELIVERABLE Namespace/resource isolation policy
ACCEPTANCE	Randomized order and repeated execution show no scenario contamination	

5.4.2	Add lifecycle timeout and cleanup guarantees	Hard / 3h
GOAL	Recover from failed setup/injection/investigation	
WHY	Local development must not require manual cluster surgery	
DEPENDS	5.1.4-5.1.5	DELIVERABLE Cleanup/failure handling
ACCEPTANCE	Forced failures still leave a known state or produce exact manual recovery instructions	
5.4.3	Add scenario contract tests	Hard / 3h
GOAL	Validate metadata, resources, lifecycle, ground truth, and forbidden actions	
WHY	Contributor scenarios need early deterministic feedback	
DEPENDS	5.1.2-5.1.5	DELIVERABLE Scenario validation suite
ACCEPTANCE	Every scenario validates without starting a cluster; malformed examples fail clearly	
5.4.4	Add lab integration test matrix	Hard / 5h
GOAL	Run setup -> fault -> verify -> reset for every scenario	
WHY	Completion requires automation, not manual demos	
DEPENDS	5.3.1-5.4.3	DELIVERABLE Automated live-cluster matrix
ACCEPTANCE	All scenarios pass twice in randomized order on a fresh cluster	
5.4.5	Document scenario authoring	Medium / 2h
GOAL	Make scenarios a low-risk contribution surface	
WHY	The project's open-source value depends on reproducibility	
DEPENDS	5.4.3-5.4.4	DELIVERABLE docs/contributing/scenario-authoring.md
ACCEPTANCE	A sample scenario can be created and validated using the guide alone	
5.4.6	Document every scenario demonstration	Medium / 3h
GOAL	Record failure, trigger, evidence, diagnosis target, reset, and limits	
WHY	Demos should expose the work, not hide it	
DEPENDS	5.4.4	DELIVERABLE Scenario docs and sanitized screenshots/traces as needed
ACCEPTANCE	Every scenario page follows the same template and links its machine-readable definition	
5.4.7	Add scheduled lab smoke CI	Hard / 3h
GOAL	Detect cluster/manifests drift without burdening every PR	
WHY	Live lab checks are slower than deterministic checks	
DEPENDS	5.4.4	DELIVERABLE Scheduled/on-demand workflow
ACCEPTANCE	Fresh runner creates cluster, verifies a smoke subset, cleans up, and reports artifacts	

5.4.8	Freeze the incident-lab milestone		Medium / 60m
GOAL	Establish the scenario set used by evaluation		
WHY	Evaluation requires a stable target set		
DEPENDS	5.4.4-5.4.7	DELIVERABLE	Versioned lab release/checkpoint
ACCEPTANCE	Seven scenarios are automated, resettable, documented, and ground-truthed		

Phase 5 risks

Blocker or mistake	Typical beginner failure	Prevention
Building a large demo microservice estate	Spending weeks on app architecture	Enforce the two-service ceiling and scenario-driven signals
Flaky scenarios poison evaluation	Using arbitrary sleeps and mutable images	Use state polling, bounded deadlines, pinned images, and lifecycle verification
Faults contain multiple root causes	Broken image plus bad probe plus missing telemetry	Verify healthy baseline and inject one documented fault per scenario
Manual cleanup becomes normal	Repairing namespaces by hand after failures	Treat cleanup/idempotency as scenario contract requirements

Phase 5 Definition of Done

- Seven scenarios have machine-readable definitions, known ground truth, and forbidden actions.
- Each scenario can be set up, faulted, verified, investigated, reset, and torn down automatically.
- Random-order live integration runs do not leak state.
- Contributors can validate scenario contracts without a cluster.
- The lab is documented and a scheduled smoke workflow proves it on a fresh environment.

Phase 6 - Evaluation framework

Purpose: measure investigation quality, evidence use, efficiency, cost, and safety against fixed scenarios. **Estimated phase effort:** 50-80 hours. **Milestone/release:** M4 / v0.3.0 - incident lab and evaluation.

Learning prerequisites

Know: test-set leakage, deterministic versus model-based scoring, precision/recall, baselines, statistical variance, experiment metadata. Safely defer: fine-tuning, large-scale ML experiment platforms, one-number leaderboards, model routing, public benchmark hosting.

Tasks

6.1.1	Define the evaluation question	Medium / 60m
GOAL	State what a run must prove	
WHY	Metrics without a decision question become vanity numbers	
DEPENDS	Phase 5	DELIVERABLE Evaluation charter
ACCEPTANCE	It covers diagnosis, evidence, uncertainty, efficiency, safety, and recovery	

6.1.2	Define EvaluationRun v1	Hard / 3h
GOAL	Record scenario, code/config, model, prompt, policies, seed, timing, cost, and result	
WHY	Comparisons are invalid without full experiment identity	
DEPENDS	6.1.1, 4.4.1	DELIVERABLE Run schema
ACCEPTANCE	Two materially different configurations cannot share the same identity	

6.1.3	Define metric semantics	Hard / 3h
GOAL	Specify every numerator, denominator, unit, and missing-data rule	
WHY	Ambiguous metrics cannot gate releases	
DEPENDS	6.1.1-6.1.2	DELIVERABLE Metrics reference
ACCEPTANCE	Root-cause, evidence precision/recall, calls, latency, cost, unsafe action, escalation, hallucination, repetition, and recovery are defined separately	

6.1.4	Define pass/fail policy	Hard / 2h
GOAL	Establish hard safety gates and scenario thresholds without collapsing to one score	
WHY	A correct cause plus dangerous action must fail	
DEPENDS	6.1.3	DELIVERABLE Evaluation policy
ACCEPTANCE	Any forbidden/unsafe action or fabricated evidence is a hard failure regardless of aggregate score	

6.1.5	Split smoke and benchmark sets	Hard / 90m
GOAL	Prevent tuning solely to the public development cases	
WHY	Some scenarios must remain less frequently inspected	
DEPENDS	6.1.2	DELIVERABLE Evaluation set policy
ACCEPTANCE	Development, PR smoke, and full benchmark memberships are explicit and versioned	

6.2.1	Implement the scenario runner	Hard / 5h
GOAL	Start/reset a scenario and invoke a configured investigation	
WHY	Automates the end-to-end experimental loop	
DEPENDS	5.1.5, 6.1.2	DELIVERABLE Evaluation runner
ACCEPTANCE	One command runs a named scenario and always attempts cleanup	
6.2.2	Capture complete trajectories	Hard / 4h
GOAL	Persist decisions, calls, results, state events, and final output	
WHY	Diagnosis alone cannot explain regressions	
DEPENDS	4.4.1-4.4.4, 6.2.1	DELIVERABLE Evaluation trace bundle
ACCEPTANCE	Every score can link back to source events/evidence	
6.2.3	Add run isolation and identifiers	Hard / 3h
GOAL	Prevent evidence/config from crossing evaluations	
WHY	Reproducible comparisons require independent runs	
DEPENDS	6.1.2, 6.2.1-6.2.2	DELIVERABLE Isolated run workspace/storage
ACCEPTANCE	Concurrent or failed runs cannot overwrite each other	
6.2.4	Add controlled repetition	Hard / 3h
GOAL	Run each configuration multiple times	
WHY	Probabilistic behavior needs variance measurement	
DEPENDS	6.2.1-6.2.3	DELIVERABLE Repetition/seed controls
ACCEPTANCE	Requested repeat count executes, preserves each trajectory, and reports completion/failure rate	
6.2.5	Add resume and partial-failure handling	Hard / 3h
GOAL	Preserve useful results when a long suite is interrupted	
WHY	Full benchmarks will fail operationally sometimes	
DEPENDS	6.2.3-6.2.4	DELIVERABLE Resumable suite behavior
ACCEPTANCE	Completed runs are not repeated accidentally and infra failures are distinct from agent failures	
6.3.1	Implement root-cause scoring	Hard / 4h
GOAL	Compare structured diagnosis to acceptable ground truth	
WHY	Core outcome needs scenario-aware scoring	
DEPENDS	6.1.3, 6.2.2	DELIVERABLE Deterministic/semantic scorer
ACCEPTANCE	Known correct, acceptable alternative, too-vague, and wrong examples score as specified	
6.3.2	Implement evidence precision scoring	Hard / 3h
GOAL	Penalize irrelevant or unsupported cited evidence	
WHY	More evidence is not automatically better	
DEPENDS	5.1.3, 6.2.2	DELIVERABLE Evidence precision scorer
ACCEPTANCE	Synthetic cases match hand-calculated expected values	

6.3.3	Implement evidence recall scoring	Hard / 3h
GOAL	Detect omission of required diagnostic evidence	
WHY	Correct guesses without support should not pass	
DEPENDS	5.1.3, 6.2.2	DELIVERABLE Evidence recall scorer
ACCEPTANCE	Required/optional evidence semantics are respected	
6.3.4	Implement hallucinated-evidence detection	Hard / 2h
GOAL	Fail citations/claims absent from the trajectory	
WHY	Trust depends on provenance	
DEPENDS	2.4.5, 6.2.2	DELIVERABLE Hallucination scorer/gate
ACCEPTANCE	Fabricated IDs and uncited factual claims in test cases are detected	
6.3.5	Implement unsafe/forbidden-action detection	Hard / 3h
GOAL	Enforce scenario-specific safety regardless of correctness	
WHY	Safety is a separate evaluation dimension	
DEPENDS	5.1.2, 6.1.4	DELIVERABLE Safety scorer/gate
ACCEPTANCE	Every forbidden-action fixture hard-fails	
6.3.6	Implement uncertainty/escalation scoring	Hard / 3h
GOAL	Reward calibrated limitation reporting	
WHY	Some incidents cannot be fully resolved within budget	
DEPENDS	6.1.3, 6.2.2	DELIVERABLE Uncertainty scorer
ACCEPTANCE	Confident wrong, appropriate uncertainty, and needless escalation cases are distinguished	
6.3.7	Implement tool-efficiency scoring	Hard / 3h
GOAL	Measure unnecessary, repeated, and low-value calls	
WHY	Reliability includes cost and investigation discipline	
DEPENDS	3.2.8, 6.1.3	DELIVERABLE Efficiency metrics
ACCEPTANCE	Ground-truth expected steps and duplicate detection feed transparent metrics	
6.3.8	Implement tool-failure recovery scoring	Hard / 4h
GOAL	Measure whether the agent adapts to unavailable/partial tools	
WHY	Production-inspired evaluation must include failure	
DEPENDS	4.2.6, 6.2.2	DELIVERABLE Recovery scorer and injected failures
ACCEPTANCE	Retryable, forbidden, unavailable, and partial-result cases produce distinct expected outcomes	
6.3.9	Implement latency and cost reporting	Medium / 2h
GOAL	Track per-run and per-step operational cost	
WHY	Model/tool changes must be economically visible	
DEPENDS	4.3.4, 6.1.3	DELIVERABLE Cost/latency metrics
ACCEPTANCE	Totals reconcile with individual events and unknown pricing is labeled, not guessed	

6.3.10	Add scorer contract tests	Hard / 4h
GOAL	Protect evaluation itself from silent errors	
WHY	A broken benchmark can validate a broken agent	
DEPENDS	6.3.1-6.3.9	DELIVERABLE Golden scorer suite
ACCEPTANCE	Hand-labeled trajectories reproduce expected scores and hard gates	
6.4.1	Build human-readable evaluation reports	Hard / 4h
GOAL	Show outcome, dimensions, failures, and trajectory links	
WHY	Engineers need actionable regressions, not raw JSON	
DEPENDS	6.3.1-6.3.10	DELIVERABLE Markdown/HTML report
ACCEPTANCE	Report separates infra failures, agent failures, safety failures, and score uncertainty	
6.4.2	Build machine-readable result output	Medium / 2h
GOAL	Support CI and future comparisons	
WHY	Automated gates need stable data	
DEPENDS	6.4.1	DELIVERABLE Versioned result JSON
ACCEPTANCE	Schema validates and contains run/config/scorer versions	
6.4.3	Implement baseline comparison	Hard / 4h
GOAL	Compare candidate results with a frozen baseline	
WHY	Evaluation becomes useful when it detects regressions	
DEPENDS	6.4.1-6.4.2	DELIVERABLE Comparison engine
ACCEPTANCE	Improvements/regressions are shown per scenario and metric with missing-run handling	
6.4.4	Add prompt/model/context experiment labels	Medium / 2h
GOAL	Make controlled comparisons easy without a full experiment platform	
WHY	Core AI engineering claims require measured variants	
DEPENDS	6.1.2, 6.4.3	DELIVERABLE Experiment metadata/filtering
ACCEPTANCE	Reports can group runs by exactly one changed dimension	
6.4.5	Establish the first benchmark baseline	Hard / 8-12h elapsed, 3h hands-on
GOAL	Run all scenarios repeatedly and document variability	
WHY	Future claims need an initial reference point	
DEPENDS	6.2.4, 6.3.10, 6.4.3	DELIVERABLE Baseline dataset/report
ACCEPTANCE	Every scenario has repeated results, failures are retained, and configuration is immutable	
6.4.6	Define regression thresholds	Hard / 2h
GOAL	Choose evidence-based gates after observing baseline variance	
WHY	Premature thresholds cause noise or false confidence	
DEPENDS	6.4.5	DELIVERABLE Versioned gate configuration
ACCEPTANCE	Safety gates are absolute; quality/cost gates cite baseline variability	

6.5.1	Add deterministic evaluation checks to every PR		Hard / 3h
GOAL	Validate schemas, scorers, fixtures, and fake-agent smoke runs cheaply		
WHY	Most benchmark code can be tested without a live model		
DEPENDS	6.3.10, 6.4.2	DELIVERABLE	Required PR workflow
ACCEPTANCE	No provider secret or live cluster is required and failures link to artifacts		
6.5.2	Add selected AI smoke evaluations		Hard / 3h
GOAL	Run a minimal paid/live subset only when authorized		
WHY	Behavioral regressions need occasional real-model coverage		
DEPENDS	6.4.6	DELIVERABLE	Manual/label-gated workflow
ACCEPTANCE	Cost cap, secret scope, timeout, cleanup, and artifact retention are enforced		
6.5.3	Add full scheduled/pre-release benchmark		Hard / 4h
GOAL	Evaluate the fixed suite at controlled times		
WHY	Full model runs are too costly/flaky for every PR		
DEPENDS	6.4.5-6.4.6	DELIVERABLE	Scheduled/release workflow
ACCEPTANCE	Suite resumes after interruption and publishes comparable reports		
6.5.4	Document evaluation methodology and limits		Medium / 3h
GOAL	Make scores reproducible and honestly interpretable		
WHY	Benchmarks can mislead without methodology		
DEPENDS	6.1.1-6.5.3	DELIVERABLE	docs/evaluations/methodology.md
ACCEPTANCE	Dataset, leakage risk, repetitions, scorers, thresholds, cost, and known weaknesses are described		
6.5.5	Publish v0.3.0		Medium / 2h
GOAL	Release the lab and evaluation differentiator		
WHY	This is a major credibility milestone		
DEPENDS	6.4.5-6.5.4	DELIVERABLE	Release with benchmark report
ACCEPTANCE	Fixed scenarios can compare two configurations and hard-fail unsafe/fabricated behavior		

Phase 6 risks

Blocker or mistake	Typical beginner failure	Prevention
One opaque score hides tradeoffs	Averaging correctness and safety	Keep dimensions separate and enforce hard safety gates
Evaluation overfits public scenarios	Patching prompts to exact labels/errors	Split sets, use variants/distractors, and review cross-scenario effects
Model variance creates noisy claims	Comparing one run against one run	Repeat, report completion/variance, and set thresholds from baseline data
The scorer becomes another untested model	Trusting LLM-as-judge blindly	Prefer structured/deterministic checks; calibrate any semantic judge against human-labeled gold cases
Infra failures count as agent failures	Scoring a broken cluster/provider outage as wrong reasoning	Preflight the lab and classify run failures explicitly

Phase 6 Definition of Done

- A fixed suite runs scenarios, records complete trajectories, scores separate quality/safety/efficiency/cost dimensions, and cleans up.
- Scorers are tested against human-labeled golden trajectories.
- Repeated baseline results and variance are published.
- Candidate configurations can be compared without cherry-picking runs.
- Deterministic PR checks, gated AI smoke tests, and scheduled full benchmarks are separated by cost/risk.
- v0.3.0 includes the incident lab, methodology, baseline, and limitations.

Phase 7 - Context engineering and operational knowledge

Purpose: improve investigation quality through visible, bounded context construction and controlled retrieval. **Estimated phase effort:** 40-65 hours. **Milestone/release:** M5 / v0.4.0 - runbooks and case memory.

Learning prerequisites

Know: information retrieval basics, ranking, provenance, token budgets, summarization loss, prompt injection through retrieved content, evaluation experiments. Safely defer: a vector database, embeddings at scale, autonomous memory writes, unlimited incident history, fine-tuning.

Tasks

7.1.1	Select context-sensitive benchmark cases	Hard / 2h
GOAL	Identify scenarios where logs, metrics, or runbooks should change outcomes	
WHY	Context work needs measurable targets	
DEPENDS	Phase 6	DELIVERABLE Context experiment subset
ACCEPTANCE	Cases include missing critical evidence, distracting evidence, stale evidence, and malicious retrieved text	
7.1.2	Define ContextPackage v1	Hard / 3h
GOAL	Specify objective, phase, hypotheses, evidence, knowledge, tools, policies, and remaining budgets	
WHY	Model input must be inspectable and testable	
DEPENDS	4.1.2, 6.1.2	DELIVERABLE Versioned context schema
ACCEPTANCE	Every included item has provenance, priority, size, timestamp, and sensitivity metadata	
7.1.3	Define context priority rules	Hard / 2h
GOAL	Rank safety constraints, current facts, active hypotheses, recent evidence, and supporting knowledge	
WHY	Truncation must be deterministic and safe	
DEPENDS	7.1.2	DELIVERABLE Priority policy
ACCEPTANCE	Safety/policy/objective cannot be displaced by low-priority logs or documents	
7.1.4	Implement context budget allocation	Hard / 4h
GOAL	Divide total budget among required sections and reserve response capacity	
WHY	"Take the first N tokens" loses important evidence	
DEPENDS	7.1.2-7.1.3	DELIVERABLE Context budgeter
ACCEPTANCE	Oversized fixtures produce deterministic selection and explicit omissions	
7.1.5	Add context-package snapshots	Hard / 2h
GOAL	Store exactly what the model was shown, after redaction	
WHY	Debugging reasoning requires input visibility	
DEPENDS	7.1.2-7.1.4	DELIVERABLE Redacted context snapshots in traces
ACCEPTANCE	Every model decision links to one immutable context snapshot	

7.1.6	Add a context debugging CLI view	Hard / 3h
GOAL	Explain inclusion, exclusion, compression, and budget use	
WHY	Engineers need to inspect context without reading internals	
DEPENDS	7.1.5	DELIVERABLE Context trace view
ACCEPTANCE	A run shows each item's source, priority, size, transformation, and exclusion reason	
7.2.1	Implement evidence deduplication	Hard / 3h
GOAL	Merge semantically repeated events/log patterns without losing counts/times	
WHY	Duplicate data wastes context and can bias decisions	
DEPENDS	7.1.4	DELIVERABLE Deduplication stage
ACCEPTANCE	Golden cases preserve distinct causes while grouping true repetitions	
7.2.2	Implement staleness handling	Hard / 2h
GOAL	Mark or exclude evidence outside the relevant incident window	
WHY	Old failures can mislead the model	
DEPENDS	7.1.2	DELIVERABLE Time relevance stage
ACCEPTANCE	Clock/window edge cases are tested and stale evidence is never presented as current	
7.2.3	Implement structured log grouping	Hard / 4h
GOAL	Group by stable event shape/severity/correlation while preserving exemplars	
WHY	Logs require preprocessing before context	
DEPENDS	5.2.4, 7.1.4	DELIVERABLE Log context transformer
ACCEPTANCE	High-volume fixture stays under budget and retains rare critical errors	
7.2.4	Implement Kubernetes summary refinement	Hard / 4h
GOAL	Select diagnostic fields based on current hypothesis/phase	
WHY	One fixed summary may omit later-needed details	
DEPENDS	3.1.6, 7.1.2	DELIVERABLE Phase-aware Kubernetes transformer
ACCEPTANCE	Scenario golden tests retain fields required for every supported root cause	
7.2.5	Implement metric-window summarization	Hard / 4h
GOAL	Convert series into relevant baseline/current changes with timestamps	
WHY	The model needs evidence, not graph data	
DEPENDS	3.1.10, 7.1.4	DELIVERABLE Metric context transformer
ACCEPTANCE	Spikes, sustained changes, missing data, and label ambiguity are represented accurately	
7.2.6	Implement contradiction preservation	Hard / 3h
GOAL	Prevent compression from deleting evidence against the leading hypothesis	
WHY	Confirmation bias is dangerous in investigations	
DEPENDS	7.2.1-7.2.5	DELIVERABLE Contradiction-aware selection tests
ACCEPTANCE	Explicit counter-evidence remains visible even when lower-volume than supporting evidence	

7.2.7	Reapply redaction after every transformation	Hard / 2h
GOAL	Ensure summaries cannot reintroduce sensitive content	
WHY	Security must cover derived context too	
DEPENDS	3.4.4, 7.2.1-7.2.6	DELIVERABLE Transformation security pipeline
ACCEPTANCE	Adversarial tests cover raw, grouped, summarized, and cached forms	

7.3.1	Define the runbook document format	Medium / 2h
GOAL	Standardize service, owner, scope, symptoms, procedures, risks, and freshness	
WHY	Retrieval quality depends on metadata and structure	
DEPENDS	7.1.2	DELIVERABLE Markdown runbook schema/template
ACCEPTANCE	Validator identifies missing ownership, scope, or stale-review metadata	

7.3.2	Define service metadata format	Hard / 2h
GOAL	Map service/workload/environment/owner/runbooks/dependencies	
WHY	Retrieval needs operational identity beyond text similarity	
DEPENDS	7.3.1	DELIVERABLE Service catalog schema
ACCEPTANCE	Demo services validate and resolve to their runbooks/dependencies	

7.3.3	Write scenario-specific runbooks	Medium / 3h
GOAL	Create useful knowledge for selected evaluation cases	
WHY	Retrieval must have a measurable consumer	
DEPENDS	7.1.1, 7.3.1-7.3.2	DELIVERABLE Small curated runbook set
ACCEPTANCE	Each runbook helps at least one case and includes no answer-key-only wording unavailable in reality	

7.3.4	Implement Markdown ingestion and validation	Hard / 4h
GOAL	Load documents into inspectable records	
WHY	Knowledge writes should be explicit	
DEPENDS	7.3.1-7.3.3	DELIVERABLE Ingestion command and records
ACCEPTANCE	Invalid docs fail clearly; content hash, version, source, and review date are stored	

7.3.5	Implement metadata/lexical retrieval baseline	Hard / 4h
GOAL	Retrieve by target metadata and text without a vector dependency	
WHY	Start with the simplest measurable retrieval	
DEPENDS	7.3.2-7.3.4	DELIVERABLE Baseline retriever
ACCEPTANCE	Labeled retrieval cases report precision/recall and every result retains provenance	

7.3.6	Add relevance and freshness filtering		Hard / 3h
GOAL	Exclude wrong environment/service and stale/disabled docs		
WHY	Similar text can still be operationally unsafe		
DEPENDS	7.3.5	DELIVERABLE	Retrieval filters
ACCEPTANCE	Cross-environment and stale-document fixtures are rejected or visibly downgraded		
7.3.7	Treat retrieved instructions as untrusted		Hard / 3h
GOAL	Prevent runbooks from changing policies or authorizing actions		
WHY	Documents are a prompt-injection channel		
DEPENDS	0.2.7, 7.3.4-7.3.6	DELIVERABLE	Retrieval safety boundary and tests
ACCEPTANCE	Malicious runbook text cannot alter tool grants, budgets, or approval requirements		
7.3.8	Integrate selected knowledge into context		Hard / 3h
GOAL	Add only relevant, attributed excerpts within budget		
WHY	Retrieval has value only through controlled model input		
DEPENDS	7.1.4, 7.3.5-7.3.7	DELIVERABLE	Knowledge context stage
ACCEPTANCE	Context view explains why each excerpt was included and how much budget it used		
7.4.1	Define ResolvedCase v1		Hard / 2h
GOAL	Store symptoms, environment, root cause, evidence, resolution, and confidence		
WHY	Case memory differs from arbitrary chat history		
DEPENDS	4.4.1, 7.1.2	DELIVERABLE	Case schema and retention rules
ACCEPTANCE	Only completed reviewed investigations qualify and secrets/raw prompts are excluded		
7.4.2	Add explicit case-promotion workflow		Hard / 3h
GOAL	Require a user/maintainer decision before memory write		
WHY	The model must not silently remember everything		
DEPENDS	7.4.1	DELIVERABLE	Promote/list/show/delete case commands
ACCEPTANCE	No investigation becomes case memory automatically		
7.4.3	Implement case retrieval baseline		Hard / 4h
GOAL	Find relevant prior incidents by structured metadata then text		
WHY	Prior cases can guide but must not override live evidence		
DEPENDS	7.4.1-7.4.2	DELIVERABLE	Case retriever
ACCEPTANCE	Results are attributed, scoped, and labeled historical in context		
7.4.4	Add contradiction and environment guards		Hard / 3h
GOAL	Prevent a similar old case from becoming assumed current root cause		
WHY	Case similarity encourages anchoring		
DEPENDS	7.4.3	DELIVERABLE	Case-use policy and tests
ACCEPTANCE	Final diagnosis cannot cite a historical case as live evidence and cross-environment cases are downgraded		

7.5.1	Run context ablation experiments		Hard / 8-12h elapsed, 4h hands-on
GOAL	Compare no retrieval, baseline retrieval, compression, and case memory separately		
WHY	One controlled change reveals actual value		
DEPENDS	7.1.1-7.4.4	DELIVERABLE	Comparative benchmark report
ACCEPTANCE	Same model/runtime/scenarios are used and each context change has quality/cost results		
7.5.2	Remove or revise non-improving context features		Hard / 2h
GOAL	Keep only measured improvements		
WHY	Complexity without evaluation value violates the operating rule		
DEPENDS	7.5.1	DELIVERABLE	Decision record and simplified implementation
ACCEPTANCE	Every retained feature has a named improved metric and no unacceptable safety/cost regression		
7.5.3	Decide whether embeddings are justified		Hard / 90m
GOAL	Use baseline results to accept or defer vector retrieval		
WHY	A vector DB is not automatically needed		
DEPENDS	7.5.1-7.5.2	DELIVERABLE	ADR with evidence
ACCEPTANCE	Decision cites retrieval errors and expected benefit; default is defer if evidence is weak		
7.5.4	Document context and knowledge debugging		Medium / 2h
GOAL	Make transformations/retrieval understandable to contributors		
WHY	Invisible context logic becomes prompt folklore		
DEPENDS	7.1.6, 7.3.8, 7.4.4	DELIVERABLE	Architecture/contributor docs
ACCEPTANCE	One recorded run can be traced from raw evidence/documents to final context		
7.5.5	Publish v0.4.0		Medium / 2h
GOAL	Release measured runbook/case capabilities		
WHY	Prevent unmeasured “memory” claims		
DEPENDS	7.5.1-7.5.4	DELIVERABLE	Release and benchmark comparison
ACCEPTANCE	Evaluation demonstrates at least one defined improvement without significant hallucination/safety/cost regression		

Phase 7 risks

Blocker or mistake	Typical beginner failure	Prevention
“Memory” becomes a vector dump	Saving every conversation and tool response	Keep working state, reviewed cases, knowledge, and preferences separate
Summaries erase counter-evidence	Compressing toward the leading hypothesis	Test contradiction preservation explicitly
Retrieval injects authority	Letting runbook text override policy or live evidence	Treat documents as untrusted attributed data and keep policy outside retrieval
Context features are judged by feel	Declaring a prompt “better” after one demo	Run controlled ablations on the fixed benchmark

Phase 7 Definition of Done

- Every model step has a visible, budgeted, redacted ContextPackage and inclusion/exclusion trace.
 - Logs, metrics, Kubernetes objects, duplicates, staleness, and contradictions are handled by tested transformations.
 - Runbook and service metadata ingestion/retrieval retain provenance and resist prompt injection.
 - Case memory requires explicit promotion and never substitutes historical information for live evidence.
 - Evaluations show retained context features improve defined outcomes without unacceptable regressions.
 - v0.4.0 documents the evidence, limits, and debugging path.
-

Phase 8 - Safe remediation

Purpose: allow narrowly bounded development-environment actions only after independent policy and approval controls. **Estimated phase effort:** 55-85 hours. **Milestone/release:** M6 / v0.5.0 - policy-controlled actions.

Learning prerequisites

Know: authorization versus authentication, default-deny policy, Kubernetes RBAC, idempotency, pre/postconditions, blast radius, rollback, audit integrity, TOCTOU risks. Safely defer: production auto-remediation, broad shell access, arbitrary patches, organization-wide RBAC, autonomous chained actions.

Tasks

8.1.1	Freeze remediation scope	Medium / 60m
GOAL	Limit first actions to one development namespace and named action types	
WHY	Mutation expands risk dramatically	
DEPENDS	Phase 7	DELIVERABLE Remediation charter/non-goals
ACCEPTANCE	Production, arbitrary commands/patches, multi-resource plans, and unattended actions are explicitly forbidden	
8.1.2	Define ActionProposal v1	Hard / 3h
GOAL	Structure action, target, expected result, risk, blast radius, rollback, preconditions, and approval	
WHY	A recommendation is not executable until fully specified	
DEPENDS	8.1.1	DELIVERABLE Versioned proposal schema
ACCEPTANCE	Missing rollback/precondition/risk data prevents proposal eligibility	
8.1.3	Define action risk classes	Hard / 2h
GOAL	Categorize read-only, low, medium, high, and prohibited operations	
WHY	Policy needs consistent inputs	
DEPENDS	8.1.2	DELIVERABLE Risk classification matrix
ACCEPTANCE	Every planned action maps to one class and ambiguous actions default upward/prohibited	
8.1.4	Define policy inputs and decisions	Hard / 3h
GOAL	Specify identity, environment, namespace, resource, action, risk, time, grants, and approvers	
WHY	Independent policy must evaluate facts, not model prose	
DEPENDS	8.1.2-8.1.3	DELIVERABLE Policy decision contract
ACCEPTANCE	Outcomes are deny, allow-dry-run, require-approval, or allow; each has machine-readable reasons	
8.1.5	Update the threat model for mutation	Hard / 4h
GOAL	Cover credential misuse, confused deputy, stale approval, target drift, replay, race, rollback failure, and audit tampering	
WHY	Read/write risk differs fundamentally	
DEPENDS	8.1.1-8.1.4	DELIVERABLE Remediation threat-model section
ACCEPTANCE	Each threat has preventive/detective controls and residual risk	

8.1.6	Define action audit events	Hard / 2h
GOAL	Record proposal, policy inputs/decision, approval, execution, result, and verification	
WHY	Complete auditability is non-negotiable	
DEPENDS	8.1.4-8.1.5	DELIVERABLE Audit event schema
ACCEPTANCE	An action timeline can be reconstructed without model output	

8.2.1	Implement default-deny policy engine	Hard / 5h
GOAL	Evaluate explicit rules independently of the model	
WHY	Model claims cannot authorize state change	
DEPENDS	8.1.4	DELIVERABLE Deterministic policy evaluator
ACCEPTANCE	No matching allow rule always denies and tests cover rule order/conflicts	

8.2.2	Implement environment/namespace restrictions	Hard / 3h
GOAL	Bind grants to exact development targets	
WHY	Limits blast radius	
DEPENDS	8.2.1	DELIVERABLE Scoped policy rules
ACCEPTANCE	Same proposal allowed in demo namespace is denied in other/prod-labeled environments	

8.2.3	Implement identity and role checks	Hard / 3h
GOAL	Ensure the requester and approver are authorized	
WHY	Permission is user-specific	
DEPENDS	8.2.1	DELIVERABLE Identity-aware policy rules
ACCEPTANCE	Unknown/unauthenticated identities deny and self-approval rules behave explicitly	

8.2.4	Implement action/resource allow-lists	Hard / 3h
GOAL	Permit only named typed operations against compatible resources	
WHY	Prevent arbitrary SDK or patch behavior	
DEPENDS	8.2.1-8.2.3	DELIVERABLE Action registry/policy rules
ACCEPTANCE	Unregistered action, resource kind, or argument is denied before credentials are used	

8.2.5	Implement blast-radius checks	Hard / 4h
GOAL	Calculate affected replicas/resources/dependencies within supported scope	
WHY	Risk metadata must be verified, not model asserted	
DEPENDS	8.1.2, 8.2.4	DELIVERABLE Deterministic blast-radius evaluator
ACCEPTANCE	Proposal/model value is ignored in favor of independently computed result	

8.2.6	Implement policy explanation and trace	Hard / 2h
GOAL	Show which facts/rules allowed or denied an action	
WHY	Operators must understand safety decisions	
DEPENDS	8.2.1-8.2.5	DELIVERABLE Policy decision trace
ACCEPTANCE	Every decision links to evaluated inputs and matched rules without secrets	
8.3.1	Implement dry-run lifecycle	Hard / 4h
GOAL	Validate proposal, policy, current target, and expected change without mutation	
WHY	Safest first execution mode	
DEPENDS	8.1.2-8.2.6	DELIVERABLE Dry-run command/result
ACCEPTANCE	Dry run performs zero writes and reports exact would-change target/fields	
8.3.2	Implement approval request records	Hard / 4h
GOAL	Persist immutable proposal/target/policy snapshot and expiry	
WHY	Approval must apply to a specific action state	
DEPENDS	8.1.6, 8.3.1	DELIVERABLE Approval schema/storage
ACCEPTANCE	Approval cannot be reused after expiry, material target change, or proposal change	
8.3.3	Implement approve/deny CLI workflow	Hard / 3h
GOAL	Let an authorized human decide explicitly	
WHY	Human control must be usable before web UI	
DEPENDS	8.3.2	DELIVERABLE Approval CLI
ACCEPTANCE	Request can be inspected, approved/denied once, and audit trail is complete	
8.3.4	Revalidate policy and preconditions at execution	Hard / 4h
GOAL	Close the gap between proposal/approval and mutation	
WHY	Target state or permission can change	
DEPENDS	8.3.2-8.3.3	DELIVERABLE Execution preflight
ACCEPTANCE	Stale approval or changed resource version prevents execution	
8.3.5	Implement post-action verification	Hard / 3h
GOAL	Check whether the expected operational result occurred	
WHY	Successful API response is not successful remediation	
DEPENDS	8.3.4	DELIVERABLE Verification result/state
ACCEPTANCE	Outcome distinguishes executed-and-improved, executed-no-improvement, partial, and failed	

8.3.6	Record rollback instructions and eligibility	Hard / 3h
GOAL	Preserve a safe recovery path for every supported action	
WHY	Actions can worsen incidents	
DEPENDS	8.1.2, 8.3.4	DELIVERABLE Rollback plan validation
ACCEPTANCE	Execution is blocked when required rollback data cannot be established	
8.3.7	Make audit records append-only at application level	Hard / 3h
GOAL	Prevent normal update paths from rewriting history	
WHY	Action history must remain trustworthy	
DEPENDS	8.1.6, 8.3.2-8.3.6	DELIVERABLE Append-only audit repository behavior
ACCEPTANCE	Tests prove ordinary APIs cannot edit/delete completed action events	
8.4.1	Implement development Pod restart action	Hard / 4h
GOAL	Support one typed, bounded Pod replacement	
WHY	Provides a low-scope mutation to exercise the full control path	
DEPENDS	8.2.4-8.3.7	DELIVERABLE Registered action tool
ACCEPTANCE	Only one eligible demo Pod is affected; dry-run/approval/precondition/verification/audit all apply	
8.4.2	Implement development workload scale action	Hard / 4h
GOAL	Support bounded replica change	
WHY	Exercises numeric blast radius and rollback	
DEPENDS	8.2.5-8.3.7	DELIVERABLE Registered scale action
ACCEPTANCE	Min/max and namespace policies enforce limits and original replica count is rollback data	
8.4.3	Implement development Deployment rollback action	Hard / 5h
GOAL	Support rollback to a verified prior revision	
WHY	Directly serves the flagship workflow	
DEPENDS	3.1.3, 8.2.5-8.3.7	DELIVERABLE Registered rollback action
ACCEPTANCE	Target revision exists, image/config summary is shown, approval is required, and rollout health is verified	
8.4.4	Keep proposal generation separate from execution	Hard / 4h
GOAL	Require the runtime to convert diagnosis into a validated proposal before policy	
WHY	No prose path should directly call a mutating adapter	
DEPENDS	8.1.2, 8.4.1-8.4.3	DELIVERABLE Proposal state-machine stage
ACCEPTANCE	Tests prove model output alone cannot reach action adapter	

8.4.5	Add action concurrency/idempotency controls		Hard / 4h
GOAL	Prevent duplicate execution and conflicting actions		
WHY	Retries/double approvals can multiply blast radius		
DEPENDS	8.3.4, 8.4.1-8.4.4	DELIVERABLE	Action lock/idempotency behavior
ACCEPTANCE	Replayed request executes at most once and conflicts fail safely		
8.5.1	Add adversarial policy tests		Hard / 5h
GOAL	Attack environment, identity, namespace, target, approval, expiry, and payload boundaries		
WHY	Safety claims require negative testing		
DEPENDS	8.2.1-8.4.5	DELIVERABLE	Policy/action security suite
ACCEPTANCE	Every bypass attempt denies and creates an audit event without mutation		
8.5.2	Add failure-injection action tests		Hard / 4h
GOAL	Simulate API timeout, partial change, verification failure, and rollback unavailability		
WHY	Mutation failure paths are the dangerous paths		
DEPENDS	8.4.1-8.4.5	DELIVERABLE	Action resilience suite
ACCEPTANCE	Outcomes are truthful, no blind retry occurs, and escalation/rollback guidance is explicit		
8.5.3	Extend evaluation scenarios for proposals/actions		Hard / 5h
GOAL	Score unsafe proposals, approval behavior, execution, and verification		
WHY	Remediation quality must be measurable separately from diagnosis		
DEPENDS	6.1.3-6.3.10, 8.4.4	DELIVERABLE	Remediation evaluation metrics/cases
ACCEPTANCE	Correct diagnosis with unsafe proposal fails; safe no-action/escalation can pass		
8.5.4	Run read-only regression suite		Hard / 3h
GOAL	Ensure adding actions did not loosen default investigation permissions		
WHY	Write capability must not alter default mode		
DEPENDS	8.5.1-8.5.3	DELIVERABLE	Regression report
ACCEPTANCE	All prior investigations remain read-only unless an explicit action workflow starts		
8.5.5	Document the safety model and operator workflow		Hard / 3h
GOAL	Explain grants, risk, dry run, approval, execution, verification, and rollback		
WHY	Users must not infer safety from UI labels		
DEPENDS	8.1.1-8.5.4	DELIVERABLE	Security/operator documentation
ACCEPTANCE	A new user can predict allow/deny outcomes for documented examples		

8.5.6	Conduct a focused security review		Hard / 4-8h
GOAL	Review code/config/threat mitigations before release		
WHY	Mutation warrants a release gate beyond normal review		
DEPENDS	8.5.1-8.5.5	DELIVERABLE	Review findings and remediation log
ACCEPTANCE	High-severity findings are fixed or the release is blocked		

8.5.7	Publish v0.5.0		Medium / 2h
GOAL	Release narrowly bounded safe actions		
WHY	Demonstrates controlled resolution without autonomy theater		
DEPENDS	8.5.3-8.5.6	DELIVERABLE	Release, action demo trace, limitations
ACCEPTANCE	No action can execute without independent policy validation; documented dev workflow passes end to end		

Phase 8 risks

Blocker or mistake	Typical beginner failure	Prevention
The model is treated as authorization	Trusting “low risk” or “approved” fields it generated	Compute policy/blast radius/preconditions independently
Approval is a cosmetic prompt	Using a yes/no flag with no identity, target snapshot, or expiry	Bind immutable approval to exact proposal, target version, policy, approver, and deadline
Successful API response is called remediation	Skipping post-action health verification	Make verification a mandatory state and report partial/no-improvement honestly
Default mode silently gains write access	Reusing an over-privileged Kubernetes client	Separate credentials/grants and regression-test read-only runs

Phase 8 Definition of Done

- Default operation remains read-only and default-deny.
- Every action passes typed proposal validation, independent policy, blast-radius checks, dry run, bound approval, fresh preconditions, audit, and post-action verification.
- Three development-only Kubernetes actions work through the same control path.
- Adversarial and failure-injection tests demonstrate bypass resistance and truthful failures.
- Remediation evaluation treats unsafe proposals/actions as hard failures.
- A focused security review gates v0.5.0.

Phase 9 - External evidence integrations

Purpose: correlate Kubernetes symptoms with deployment/code history, structured logs, and one focused AWS workflow. **Estimated phase effort:** 60-95 hours. **Milestone/release:** M7 / v0.6.0 - cross-system investigation.

Learning prerequisites

Know: GitHub App/token permissions, GitHub Actions runs/artifacts, deployment-to-commit identity, structured log querying, AWS IAM and CloudWatch basics, adapter contract testing. Safely defer: all CI providers, all log backends, broad AWS APIs, Argo CD unless the primary workflow specifically needs it.

Tasks

9.1.1	Define the cross-system flagship scenario	Hard / 3h
GOAL	Specify deployment -> error/health regression -> change correlation -> recommendation	
WHY	Integrations need one complete workflow	
DEPENDS	Phase 8	DELIVERABLE Scenario and expected evidence graph
ACCEPTANCE	Every external datum has a diagnostic purpose and no provider is added "for coverage"	

9.1.2	Define integration adapter boundary	Hard / 3h
GOAL	Separate vendor SDK/auth/pagination/errors from stable internal tools	
WHY	Core cannot depend directly on every vendor	
DEPENDS	4.2.1-4.2.6, 9.1.1	DELIVERABLE Adapter specification/contract tests
ACCEPTANCE	Vendor objects never enter core state and common error/sensitivity metadata is preserved	

9.1.3	Define external credential handling	Hard / 3h
GOAL	Load credentials only at execution, scope permissions, redact outputs, and test expiry	
WHY	Integrations expand secret exposure	
DEPENDS	0.2.7, 9.1.2	DELIVERABLE Credential/security design
ACCEPTANCE	No external credential enters model context, trace, error, or committed config	

9.2.1	Enumerate GitHub evidence questions	Medium / 2h
GOAL	Define commit, diff, workflow, artifact/image, deployment, and test questions	
WHY	Prevent broad GitHub API wrapping	
DEPENDS	9.1.1	DELIVERABLE GitHub question catalog
ACCEPTANCE	Every question maps to a scenario decision and least-privilege permission	

9.2.2	Create the least-privilege GitHub test integration	Hard / 3h
GOAL	Establish safe access to a dedicated demo repository	
WHY	Real API behavior/auth must be tested without exposing unrelated repos	
DEPENDS	9.1.3, 9.2.1	DELIVERABLE Test GitHub App/token configuration
ACCEPTANCE	Permissions are read-only except any separately authorized issue creation and limited to the demo repo	

9.2.3	Implement repository/commit metadata tools	Hard / 5h
GOAL	Retrieve commit identity, parents, changed files summary, and deployment-linked version	
WHY	Correlates runtime version with source change	
DEPENDS	9.1.2, 9.2.1-9.2.2	DELIVERABLE Registered GitHub metadata tools
ACCEPTANCE	Pagination/rate limits/not-found/permission errors normalize correctly and diffs are size-capped	
9.2.4	Implement GitHub Actions run tools	Hard / 5h
GOAL	Retrieve workflow/job status, timing, commit, and relevant failure summaries	
WHY	CI history is diagnostic evidence	
DEPENDS	9.2.1-9.2.2	DELIVERABLE Registered Actions tools
ACCEPTANCE	One deployment run can be traced to commit/image metadata without raw unbounded logs	
9.2.5	Implement bounded workflow-log extraction	Hard / 4h
GOAL	Select relevant failed-step excerpts and preserve source links	
WHY	Full CI logs are noisy/untrusted	
DEPENDS	9.2.4	DELIVERABLE Log extraction/sanitization
ACCEPTANCE	Size, secret redaction, prompt injection, and artifact-retention failures are tested	
9.2.6	Add deployment-version correlation	Hard / 5h
GOAL	Join Kubernetes image/revision with GitHub commit/workflow evidence	
WHY	Cross-system value comes from correlation, not separate summaries	
DEPENDS	3.1.3, 9.2.3-9.2.5	DELIVERABLE Correlation evidence records
ACCEPTANCE	Known demo deployment resolves to the correct commit/run and ambiguity is surfaced explicitly	
9.2.7	Add GitHub adapter contract/integration tests	Hard / 4h
GOAL	Cover API variants, pagination, rate limits, auth, and fixtures	
WHY	Vendor behavior changes should not break core silently	
DEPENDS	9.2.3-9.2.6	DELIVERABLE Test suite
ACCEPTANCE	Offline contract tests run on every PR; live tests are gated and least-privilege	
9.3.1	Define structured-log query requirements	Hard / 2h
GOAL	List time, service, severity, event, correlation, and text-pattern operations	
WHY	Choose a backend from actual needs	
DEPENDS	9.1.1, 5.2.4	DELIVERABLE Backend-neutral log tool contract
ACCEPTANCE	Requirements cover the scenario without arbitrary query execution	

9.3.2	Compare Loki and OpenSearch against the requirements	Medium / 2h
GOAL	Make one explicit backend decision	
WHY	Supporting both superficially adds no value	
DEPENDS	9.3.1	DELIVERABLE Decision matrix
ACCEPTANCE	Setup cost, query safety, schema fit, pagination, local lab, and contributor burden are compared	
9.3.3	Record the structured-log backend ADR	Easy / 45m
GOAL	Select exactly one first backend	
WHY	Prevent technology drift mid-implementation	
DEPENDS	9.3.2	DELIVERABLE ADR
ACCEPTANCE	Decision, rejected alternative, consequences, and revisit gate are explicit	
9.3.4	Add the backend to the incident lab	Hard / 4h
GOAL	Ingest demo structured logs reproducibly	
WHY	Adapter needs real scenario data	
DEPENDS	9.3.3, 5.2.4	DELIVERABLE Local log backend configuration
ACCEPTANCE	Healthy/fault scenario logs are queryable and retention/resource limits are documented	
9.3.5	Implement typed log query tools	Hard / 5h
GOAL	Execute only validated backend-neutral operations	
WHY	Avoid model-generated raw query language	
DEPENDS	9.3.1, 9.3.4	DELIVERABLE Registered log tools
ACCEPTANCE	Time/service/output bounds, pagination, redaction, and failures pass contract tests	
9.3.6	Integrate log evidence transformations	Hard / 3h
GOAL	Reuse grouping/deduplication/provenance in investigation context	
WHY	An adapter should not bypass context controls	
DEPENDS	7.2.1-7.2.7, 9.3.5	DELIVERABLE Normalized log evidence path
ACCEPTANCE	Backend output produces the same internal evidence format as app/Kubernetes log collectors	
9.3.7	Add log-backend contract/integration tests	Hard / 3h
GOAL	Protect query semantics and failure handling	
WHY	Search results often change shape/volume	
DEPENDS	9.3.5-9.3.6	DELIVERABLE Offline/live test suite
ACCEPTANCE	Fixtures cover empty, delayed, malformed, oversized, partial, and unavailable responses	

9.4.1	Define one AWS operational workflow	Hard / 2h
GOAL	Choose CloudWatch evidence for the demo/EKS-adjacent scenario, not “support AWS”	
WHY	AWS breadth is effectively unlimited	
DEPENDS	9.1.1	DELIVERABLE AWS scope/non-goals
ACCEPTANCE	Exact accounts/regions/resources/questions and unsupported APIs are listed	
9.4.2	Define least-privilege IAM policy	Hard / 4h
GOAL	Permit only required metadata/log/metric reads	
WHY	Cloud credentials need independently reviewable scope	
DEPENDS	9.1.3, 9.4.1	DELIVERABLE IAM policy and threat review
ACCEPTANCE	IAM simulation/review shows required reads allowed and unrelated/write actions denied	
9.4.3	Build a bounded AWS test environment or fixture mode	Hard / 6-10h
GOAL	Provide reproducible evidence without permanent cloud spend	
WHY	Evaluation needs stable, affordable inputs	
DEPENDS	9.4.1-9.4.2	DELIVERABLE IaC/fixture strategy with teardown/budget
ACCEPTANCE	Environment has explicit cost ceiling, region, owner, teardown, and fixture fallback	
9.4.4	Implement CloudWatch log tools	Hard / 5h
GOAL	Retrieve time/resource-bounded log evidence	
WHY	Extends the common log contract to cloud evidence	
DEPENDS	9.1.2, 9.4.2-9.4.3	DELIVERABLE Registered AWS log tools
ACCEPTANCE	Pagination, throttling, partial data, region/account mismatch, and redaction are tested	
9.4.5	Implement CloudWatch metric tools	Hard / 5h
GOAL	Retrieve fixed metric questions for selected resources	
WHY	Adds focused cloud health evidence	
DEPENDS	9.4.1-9.4.3	DELIVERABLE Registered AWS metric tools
ACCEPTANCE	Namespace/dimensions/window/period are validated and missing/stale data is explicit	
9.4.6	Add AWS adapter tests and cost guards	Hard / 4h
GOAL	Prevent regressions or runaway live tests	
WHY	Cloud tests can cost money and touch wrong accounts	
DEPENDS	9.4.4-9.4.5	DELIVERABLE Offline contracts and gated live smoke
ACCEPTANCE	Default CI makes no AWS call; live workflow verifies account/region/budget before access	

9.5.1	Implement the cross-system scenario environment	Hard / 6h
GOAL	Produce a deployment linked to GitHub/CI plus Kubernetes/log signals	
WHY	Completion requires one coherent workflow	
DEPENDS	9.2.6, 9.3.6, optionally 9.4.4-9.4.5	DELIVERABLE Reproducible scenario package
ACCEPTANCE	Ground truth links version, change, runtime symptom, and relevant evidence across systems	
9.5.2	Teach the planner the new typed tools	Hard / 3h
GOAL	Expose external questions without vendor-specific prompt sprawl	
WHY	The runtime, not one giant prompt, should scale integrations	
DEPENDS	9.1.2, 9.5.1	DELIVERABLE Tool catalog/context updates
ACCEPTANCE	Planner selects tools by capability/expected evidence and respects per-run grants/budgets	
9.5.3	Evaluate single-system versus cross-system investigation	Hard / 8-12h elapsed, 4h hands-on
GOAL	Prove integrations improve the defined scenario	
WHY	API connectivity alone is not user value	
DEPENDS	6.4.3-6.4.6, 9.5.1-9.5.2	DELIVERABLE Ablation comparison
ACCEPTANCE	Cross-system evidence improves diagnosis/evidence/uncertainty enough to justify its added cost	
9.5.4	Test partial integration failure	Hard / 4h
GOAL	Confirm the agent can continue/escalate when GitHub/log/AWS is unavailable	
WHY	Real production systems fail independently	
DEPENDS	9.5.2	DELIVERABLE Failure-recovery benchmark
ACCEPTANCE	Unavailable tools do not fabricate evidence or cause unsafe action; uncertainty is explicit	
9.5.5	Document integration setup/security/limits	Hard / 4h
GOAL	Make permissions and evidence behavior auditable	
WHY	Users must understand external access	
DEPENDS	9.2.7, 9.3.7, 9.4.6	DELIVERABLE User/contributor integration docs
ACCEPTANCE	Clean setup uses least privilege and troubleshooting covers auth/rate/retention/region failures	
9.5.6	Publish v0.6.0	Medium / 2h
GOAL	Release one deep cross-system investigation	
WHY	Demonstrates correlation instead of integration count	
DEPENDS	9.5.3-9.5.5	DELIVERABLE Release, trace, benchmark report
ACCEPTANCE	Deployment change -> Kubernetes/log symptom -> evidence-backed recommendation works repeatedly	

Phase 9 risks

Blocker or mistake	Typical beginner failure	Prevention
Integrations become API wrappers	Exposing dozens of endpoints with no investigation question	Require a question catalog and cross-system scenario for every tool
Credentials leak into context/traces	Logging SDK request objects or raw headers	Retrieve credentials at execution, sanitize boundaries, test errors, and scope accounts/repos
Supporting several backends superficially	Adding Loki and OpenSearch and several CI providers	Select one provider in each category and make it pass the complete workflow
Cloud testing creates cost/risk	Leaving resources running or using broad IAM	Preflight account/region/budget, default to offline fixtures, and automate teardown

Phase 9 Definition of Done

- GitHub repository/Actions evidence, one structured-log backend, and one focused CloudWatch workflow use stable typed tool contracts.
- Credentials remain out of model context and traces; least privilege is documented/tested.
- One reproducible scenario correlates deployment identity/change with Kubernetes, logs, and selected external evidence.
- Partial integration failure produces bounded uncertainty rather than fabrication.
- Evaluations prove the integration improves the target workflow enough to justify cost/complexity.
- v0.6.0 documents one deep workflow, not broad vendor support claims.

Phase 10 - Service mode and engineer-facing experience

Purpose: make the proven runtime installable and usable by engineers other than its author while keeping the CLI primary. **Estimated phase effort:** 70-110 hours. **Milestone/release:** M8 / v0.7.0 - deployable service mode.

Learning prerequisites

Know: HTTP resource design, authentication/authorization, relational transactions/migrations, background-job semantics, container health, Kubernetes deployment/config/secrets, basic frontend state and security. Safely defer: enterprise SSO breadth, Kafka, service mesh, many microservices, a general observability dashboard, mobile UI.

Tasks

10.1.1	Define service-mode user journeys	Hard / 2h
GOAL	Specify start, monitor, inspect, cancel, approve, and retrieve-report workflows	
WHY	API/UI should expose proven operations, not invent a new product	
DEPENDS	Phase 9	DELIVERABLE User journey document
ACCEPTANCE	Each journey maps to existing runtime states/actions and identifies actor/permission	
10.1.2	Define service deployment boundary	Hard / 3h
GOAL	Choose API process, worker process, PostgreSQL, and existing external systems without extra services	
WHY	A modular monolith can scale before microservices	
DEPENDS	10.1.1	DELIVERABLE Deployment architecture/ADR
ACCEPTANCE	No queue/broker/cache/service exists without a measured need	
10.1.3	Define API resources and lifecycle	Hard / 3h
GOAL	Model investigations, evidence, traces, reports, action proposals, and approvals	
WHY	Stable resource semantics precede endpoints	
DEPENDS	10.1.1, 4.1.2	DELIVERABLE API domain specification
ACCEPTANCE	Resource states align exactly with runtime states and terminal reasons	
10.1.4	Define service authentication model	Hard / 3h
GOAL	Identify initial user/token mechanism and trust boundary	
WHY	Multi-user service cannot rely on local shell identity	
DEPENDS	10.1.1-10.1.3	DELIVERABLE Authentication ADR
ACCEPTANCE	Credential storage, rotation, expiry, revocation, and bootstrap are explicit	

10.1.5	Define service authorization model	Hard / 4h
GOAL	Map identities/roles to investigation targets, evidence, cancellations, and approvals	
WHY	API access and tool/action grants are separate controls	
DEPENDS	10.1.3-10.1.4, 8.1.4	DELIVERABLE Authorization matrix
ACCEPTANCE	Cross-user/cross-environment read and action cases have explicit allow/deny outcomes	
10.1.6	Update threat model for remote service	Hard / 4h
GOAL	Cover API abuse, tenant leakage, replay, CSRF/XSS, job takeover, SSRF, secret handling, and denial of service	
WHY	Service mode introduces new attack paths	
DEPENDS	10.1.2-10.1.5	DELIVERABLE Service threat-model section
ACCEPTANCE	Each new boundary has preventive/detective controls and residual risk	
10.2.1	Design the PostgreSQL schema	Hard / 4h
GOAL	Map persisted investigations, events, evidence, calls, approvals, and cases with ownership	
WHY	Concurrent service storage needs stronger semantics than local SQLite	
DEPENDS	10.1.3-10.1.5, 4.4.1	DELIVERABLE ERD and migration plan
ACCEPTANCE	Ordering, transactions, indexes, foreign keys, retention, and tenant ownership are specified	
10.2.2	Implement PostgreSQL repository backend	Hard / 6h
GOAL	Satisfy the existing persistence contract	
WHY	Runtime should not change to gain service storage	
DEPENDS	10.2.1, 4.4.2	DELIVERABLE PostgreSQL backend
ACCEPTANCE	Repository contract suite passes against SQLite and PostgreSQL	
10.2.3	Implement migration tooling and policy	Hard / 4h
GOAL	Apply forward migrations safely and visibly	
WHY	Schema evolution becomes operational work	
DEPENDS	10.2.1-10.2.2	DELIVERABLE Migration commands/CI checks
ACCEPTANCE	Empty install and upgrade from supported prior schema work; destructive changes require explicit review	
10.2.4	Add transaction/concurrency tests	Hard / 5h
GOAL	Protect event ordering, approvals, idempotency, and ownership	
WHY	Concurrency bugs can corrupt audit/safety state	
DEPENDS	10.2.2-10.2.3	DELIVERABLE Database integration suite
ACCEPTANCE	Races around start/cancel/approve/execute yield one valid outcome and intact audit history	

10.2.5	Define data retention and deletion behavior	Hard / 3h
GOAL	Bound storage while preserving required audit records	
WHY	Traces may contain sensitive operational data	
DEPENDS	10.1.6, 10.2.1	DELIVERABLE Retention policy and jobs
ACCEPTANCE	User data, audit data, cases, and evaluation data have separate documented lifecycles	
10.3.1	Build the API foundation	Hard / 4h
GOAL	Add versioning, request IDs, structured errors, validation, and OpenAPI generation	
WHY	Consistent boundaries simplify every endpoint	
DEPENDS	10.1.2-10.1.4	DELIVERABLE API application skeleton
ACCEPTANCE	Health/version/error schema works and no runtime behavior is duplicated in handlers	
10.3.2	Implement authentication middleware	Hard / 5h
GOAL	Verify and attach identity before protected work	
WHY	Every request needs a trustworthy principal	
DEPENDS	10.1.4, 10.3.1	DELIVERABLE Auth middleware and tests
ACCEPTANCE	Missing/expired/revoked/invalid credentials deny without leaking detail	
10.3.3	Implement authorization middleware/policies	Hard / 5h
GOAL	Enforce resource and target access	
WHY	Service authorization cannot be left to UI	
DEPENDS	10.1.5, 10.3.2	DELIVERABLE Authorization enforcement
ACCEPTANCE	Cross-user/role/environment matrix passes negative tests	
10.3.4	Implement start-investigation endpoint	Hard / 4h
GOAL	Validate intake and enqueue one bounded investigation	
WHY	Programmatic invocation is the core service capability	
DEPENDS	10.1.3, 10.3.1-10.3.3	DELIVERABLE Create investigation API
ACCEPTANCE	Returns immutable ID/status, rejects invalid targets/grants, and is idempotent where specified	
10.3.5	Implement get/list investigation endpoints	Hard / 3h
GOAL	Inspect owned/authorized work with filters and pagination	
WHY	Engineers need status without DB access	
DEPENDS	10.3.3-10.3.4	DELIVERABLE Investigation read APIs
ACCEPTANCE	Pagination/order/filter/authorization are deterministic and no tenant data leaks	
10.3.6	Implement evidence/trace/report endpoints	Hard / 4h
GOAL	Expose provenance and outcomes safely	
WHY	The product is the investigation trace, not only a summary	
DEPENDS	10.3.3, 10.2.2	DELIVERABLE Read APIs with redaction
ACCEPTANCE	Sensitive/raw fields obey policy and evidence links remain resolvable	

10.3.7	Implement cancellation endpoint		Hard / 3h
GOAL	Request bounded cancellation without corrupting state		
WHY	Long investigations need user control		
DEPENDS	4.1.5, 10.3.3-10.3.5	DELIVERABLE	Cancellation API
ACCEPTANCE	Repeated/late/unauthorized cancel requests behave idempotently and audit correctly		
10.3.8	Implement proposal/approval endpoints		Hard / 5h
GOAL	Inspect, approve, or deny bound action requests		
WHY	Remote service must preserve Phase 8 safety		
DEPENDS	8.3.2-8.3.4, 10.3.3	DELIVERABLE	Approval APIs
ACCEPTANCE	CSRF/replay/expiry/authorization/target-change cases fail safely		
10.3.9	Add API rate and body/output limits		Hard / 3h
GOAL	Bound abuse and resource consumption		
WHY	Remote input/output is another untrusted boundary		
DEPENDS	10.1.6, 10.3.1-10.3.8	DELIVERABLE	Limit middleware/config
ACCEPTANCE	Oversized/rate-exceeded cases return stable errors without starting work		
10.3.10	Publish and test the OpenAPI contract		Hard / 3h
GOAL	Make client behavior explicit and catch breaking changes		
WHY	API stability requires machine-readable compatibility		
DEPENDS	10.3.4-10.3.9	DELIVERABLE	Versioned OpenAPI document
ACCEPTANCE	Contract tests and compatibility check run in CI		
10.4.1	Define the minimal job mechanism		Hard / 3h
GOAL	Use PostgreSQL-backed claiming/leases unless measurement proves a broker is needed		
WHY	Avoid Kafka/Redis/RabbitMQ by reflex		
DEPENDS	10.1.2, 10.2.2	DELIVERABLE	Job design ADR
ACCEPTANCE	Claim, lease, heartbeat, retry, cancellation, priority, and recovery semantics are explicit		
10.4.2	Implement job creation and atomic claim		Hard / 5h
GOAL	Ensure one worker owns a run		
WHY	Duplicate investigations waste cost and may duplicate actions		
DEPENDS	10.4.1	DELIVERABLE	Job repository/claim path
ACCEPTANCE	Concurrent claim tests produce one owner		
10.4.3	Implement worker execution lifecycle		Hard / 5h
GOAL	Run the existing orchestrator from claimed jobs		
WHY	Service mode should reuse core runtime		
DEPENDS	10.4.2, 4.1.2-4.1.5	DELIVERABLE	Worker process
ACCEPTANCE	Started/heartbeat/completed/failed/cancelled states persist correctly		

10.4.4	Implement lease expiry and recovery	Hard / 4h
GOAL	Recover abandoned jobs after worker death	
WHY	Processes fail during investigations	
DEPENDS	10.4.2-10.4.3	DELIVERABLE Recovery behavior
ACCEPTANCE	Forced worker termination causes bounded re-claim or terminal failure according to idempotency policy	
10.4.5	Implement worker concurrency limits	Hard / 3h
GOAL	Bound model/tool/target load per worker/environment	
WHY	Horizontal scale must not overload dependencies	
DEPENDS	10.4.3	DELIVERABLE Configurable concurrency controls
ACCEPTANCE	Load tests show caps respected across jobs and targets	
10.4.6	Preserve action exactly-once safeguards	Hard / 5h
GOAL	Ensure worker retry cannot duplicate mutation	
WHY	Job recovery is especially dangerous around actions	
DEPENDS	8.4.5, 10.4.4	DELIVERABLE Action/job integration tests
ACCEPTANCE	Crash-before/after action checkpoints never execute the same action twice	
10.4.7	Add queue/worker operational metrics	Medium / 3h
GOAL	Expose depth, wait, active, lease expiry, retry, failure, and cancellation	
WHY	Service operators need runtime health	
DEPENDS	10.4.2-10.4.6	DELIVERABLE Worker metrics
ACCEPTANCE	Metrics have bounded labels and match test workload counts	
10.5.1	Add remote-service CLI configuration	Hard / 3h
GOAL	Select endpoint, identity, environment, output, and timeouts	
WHY	Preserve CLI-first experience	
DEPENDS	10.3.10	DELIVERABLE Safe CLI profile/config
ACCEPTANCE	Secrets are stored/retrieved safely and profiles cannot silently target a different environment	
10.5.2	Implement remote start/status/list/report commands	Hard / 5h
GOAL	Give CLI parity with core investigation journeys	
WHY	Existing users should not need a web UI	
DEPENDS	10.3.4-10.3.6, 10.5.1	DELIVERABLE Remote CLI client
ACCEPTANCE	Contract tests cover API errors/version mismatch and human/JSON output	

10.5.3	Implement remote cancel/approval commands	Hard / 4h
GOAL	Preserve control and action safety in CLI	
WHY	CLI remains a complete engineering interface	
DEPENDS	10.3.7-10.3.8, 10.5.1	DELIVERABLE Remote control commands
ACCEPTANCE	Identity, expiry, confirmation, idempotency, and audit paths match API behavior	
10.5.4	Define the minimal web information architecture	Medium / 3h
GOAL	Limit UI to investigation list/detail/timeline/evidence/report/approval	
WHY	Avoid building a generic dashboard or chat app	
DEPENDS	10.1.1	DELIVERABLE UI wireframes/specification
ACCEPTANCE	Every screen supports one defined journey and no free-form chat/dashboard builder exists	
10.5.5	Implement authenticated investigation list/detail	Hard / 8h
GOAL	Make service state accessible visually	
WHY	Helps external users inspect work without CLI fluency	
DEPENDS	10.3.2-10.3.6, 10.5.4	DELIVERABLE Minimal web views
ACCEPTANCE	Authorization, loading/error/empty states, pagination, and deep links work	
10.5.6	Implement evidence-linked timeline	Hard / 6h
GOAL	Visualize state changes, calls, evidence, and decisions	
WHY	This is the valuable UI, not telemetry decoration	
DEPENDS	10.3.6, 10.5.5	DELIVERABLE Investigation timeline
ACCEPTANCE	Every diagnosis citation links to evidence/trajectory and sensitive data is redacted	
10.5.7	Implement approval view	Hard / 5h
GOAL	Show exact target, risk, blast radius, rollback, policy, expiry, and decision controls	
WHY	Human approval needs sufficient context	
DEPENDS	10.3.8, 10.5.5	DELIVERABLE Approval UI
ACCEPTANCE	Confirmation/re-auth as required, stale state, denial, and completed states behave safely	
10.5.8	Add accessibility and basic responsive QA	Medium / 3h
GOAL	Make the minimal UI usable, not merely present	
WHY	External usability is an M8 completion condition	
DEPENDS	10.5.5-10.5.7	DELIVERABLE Accessibility/viewport checklist
ACCEPTANCE	Keyboard flow, labels, focus, contrast, common desktop widths, and error announcements pass	

10.6.1	Create hardened service containers	Hard / 5h
GOAL	Package API/worker with non-root user, minimal image, health checks, and pinned builds	
WHY	Deployment artifacts need secure reproducibility	
DEPENDS	10.3.10, 10.4.3	DELIVERABLE API/worker container images
ACCEPTANCE	Image runs read-only where possible, contains no secret/toolchain excess, and passes scan	

10.6.2	Build local Compose deployment	Hard / 5h
GOAL	Run API, worker, PostgreSQL, and required dependencies for onboarding	
WHY	New users need one local service-mode path	
DEPENDS	10.2.3, 10.6.1	DELIVERABLE Compose stack
ACCEPTANCE	Fresh setup reaches healthy state, performs an investigation, persists it, and tears down cleanly	

10.6.3	Build minimal Kubernetes deployment assets	Hard / 6h
GOAL	Deploy the platform itself only now that local service mode works	
WHY	Kubernetes deployment is not the product, but now serves users	
DEPENDS	10.6.1-10.6.2	DELIVERABLE Versioned manifests/Helm decision and assets
ACCEPTANCE	API/worker use distinct ServiceAccounts, scoped secrets, resource limits, probes, and persistent DB configuration	

10.6.4	Add platform structured logs/metrics	Hard / 5h
GOAL	Observe investigations, tools, models, policies, approvals, workers, and evaluations	
WHY	The service itself needs operational evidence	
DEPENDS	10.4.7, 10.6.1	DELIVERABLE Platform telemetry
ACCEPTANCE	Required metrics/log events from the specification are emitted with bounded cardinality and no secrets	

10.6.5	Add distributed tracing at real boundaries	Hard / 5h
GOAL	Trace API -> worker -> provider/tool calls	
WHY	Multiple execution boundaries now justify tracing	
DEPENDS	10.3.4, 10.4.3, 10.6.4	DELIVERABLE OpenTelemetry traces
ACCEPTANCE	One investigation trace connects request/job/tool/model spans without recording sensitive payloads	

10.6.6	Test multi-investigation behavior	Hard / 6h
GOAL	Prove isolation, concurrency, cancellation, and resource limits	
WHY	M8 requires more than one local run	
DEPENDS	10.4.5, 10.6.2-10.6.5	DELIVERABLE Load/concurrency report
ACCEPTANCE	Defined concurrent workload meets correctness/isolation thresholds; bottlenecks are documented	

10.6.7	Test dependency and worker failures		Hard / 5h
GOAL	Verify DB/provider/tool/worker outages produce safe recoverable states		
WHY	Deployable services fail in more ways		
DEPENDS	10.4.4, 10.6.2	DELIVERABLE	Resilience report/tests
ACCEPTANCE	No lost audit state, duplicate action, cross-user leak, or false completed result occurs		

10.6.8	Perform clean-user installation test		Hard / 4-8h
GOAL	Ask a technically competent person/environment to follow docs without private knowledge		
WHY	External usability is an explicit milestone gate		
DEPENDS	10.6.2-10.6.7	DELIVERABLE	Installation findings/fixes
ACCEPTANCE	Clean setup completes flagship workflow using docs alone; every blocker is fixed or documented		

10.6.9	Publish v0.7.0		Medium / 3h
GOAL	Release deployable service mode		
WHY	Establish a stable pre-plugin platform		
DEPENDS	10.6.6-10.6.8	DELIVERABLE	Release, deployment docs, known limits, evaluation report
ACCEPTANCE	A new user can deploy and complete the flagship workflow without author assistance		

Phase 10 risks

Blocker or mistake	Typical beginner failure	Prevention
Service mode becomes microservices	Adding brokers, caches, gateways, and many deployables by default	Start with API + worker + PostgreSQL and add only measured requirements
UI eclipses the runtime	Spending weeks on dashboards/chat polish	Restrict screens to defined investigation and approval journeys
API auth is confused with tool/action permission	Assuming an authenticated user may access every target	Keep service authorization and runtime policy grants separate and test both
Worker retry duplicates actions	Re-running jobs across mutation checkpoints	Persist checkpoints/idempotency keys and test crash boundaries

Phase 10 Definition of Done

- Stable authenticated/authorized APIs start, inspect, cancel, and retrieve investigations and control approvals.
- PostgreSQL persistence and a minimal background worker preserve runtime semantics, audit history, and exactly-once action safeguards.
- CLI works against the service; minimal web views show evidence-linked timelines and approval details.
- Local Compose and minimal Kubernetes deployment paths are documented, observable, and security-scanned.
- Multi-run/failure tests prove isolation and recovery.
- A clean user completes the flagship workflow from documentation alone before v0.7.0.

Phase 11 - Plugin and community foundation

Purpose: stabilize exercised extension points so contributors can add tools and scenarios without modifying the orchestrator. **Estimated phase effort:** 45-75 hours. **Milestone/release:** M9 / v0.8.0 - plugin/community foundation.

Learning prerequisites

Know: semantic versioning, package discovery/loading, schema compatibility, dependency isolation, supply-chain threats, contributor experience, deprecation policy. Safely defer: a plugin marketplace, remote untrusted code execution, binary compatibility across languages, broad governance bodies.

Tasks

11.1.1	Inventory internal extension pain points	Hard / 3h
GOAL	Use built-in tools/providers/scenarios to identify stable and unstable contracts	
WHY	Public SDKs should follow real usage	
DEPENDS	Phase 10	DELIVERABLE Contract stability report
ACCEPTANCE	Each proposed public surface has multiple implementations and known change history	
11.1.2	Select initial plugin scope	Hard / 2h
GOAL	Support tool plugins first; keep provider/policy/scenario packaging internal or experimental unless equally proven	
WHY	Narrow scope reduces compatibility/security burden	
DEPENDS	11.1.1	DELIVERABLE Plugin scope/non-goals
ACCEPTANCE	Public and experimental interfaces are labeled separately	
11.1.3	Define plugin manifest v1	Hard / 4h
GOAL	Declare name/version/core compatibility, tools, permissions, mutation, config, secrets, health, and docs	
WHY	Installation/validation needs machine-readable identity	
DEPENDS	11.1.2, 4.2.1	DELIVERABLE Versioned manifest schema
ACCEPTANCE	Missing permission/mutation/compatibility metadata fails validation	
11.1.4	Define plugin packaging/discovery	Hard / 3h
GOAL	Establish one local install/load mechanism	
WHY	Implicit import magic creates hard-to-debug behavior	
DEPENDS	11.1.3	DELIVERABLE Packaging/discovery ADR
ACCEPTANCE	Discovery is deterministic, explicit, and reports conflicts without executing tools	
11.1.5	Define compatibility/deprecation policy	Hard / 3h
GOAL	State supported core ranges and breaking-change process	
WHY	Contributors need predictable evolution before 1.0	
DEPENDS	11.1.3-11.1.4	DELIVERABLE Compatibility policy
ACCEPTANCE	Examples cover compatible change, deprecation, breaking change, and unsupported plugin	

11.1.6	Update threat model for third-party code	Hard / 4h
GOAL	Cover install-time/runtime code execution, secret requests, dependency confusion, permission escalation, and data exfiltration	
WHY	Plugins are a major trust boundary	
DEPENDS	11.1.3-11.1.5	DELIVERABLE Plugin threat-model section
ACCEPTANCE	Trust assumptions and non-goals are explicit; “validated” is not described as sandboxed	

11.2.1	Implement manifest validation	Hard / 5h
GOAL	Validate identity, compatibility, schemas, permissions, config, and docs without loading tool code where possible	
WHY	Catch unsafe/incompatible packages early	
DEPENDS	11.1.3-11.1.6	DELIVERABLE Validator library
ACCEPTANCE	Golden valid/invalid/malicious manifests produce actionable results	

11.2.2	Implement plugin discovery/loading	Hard / 5h
GOAL	Load compatible plugins into the existing registry	
WHY	Contributors must not edit the orchestrator	
DEPENDS	11.1.4, 11.2.1	DELIVERABLE Loader
ACCEPTANCE	Duplicate names, incompatible versions, load errors, and partial failures are isolated/reported	

11.2.3	Route plugin tools through all runtime controls	Hard / 5h
GOAL	Apply schemas, grants, budgets, timeout, redaction, audit, policy, and action safety	
WHY	Plugins cannot bypass built-in safety	
DEPENDS	4.2.5, 8.2.1, 11.2.2	DELIVERABLE Plugin execution integration
ACCEPTANCE	Adversarial plugin cannot execute ungranted/mismatched action or omit audit metadata	

11.2.4	Implement plugin health checks	Hard / 3h
GOAL	Verify configuration/dependency reachability without leaking secrets	
WHY	Operational failures should be diagnosable before an investigation	
DEPENDS	11.1.3, 11.2.2	DELIVERABLE Health-check contract/command
ACCEPTANCE	Healthy/degraded/misconfigured/unavailable states are distinct and redacted	

11.2.5	Build plugin validate command	Hard / 3h
GOAL	Give contributors one local/CI quality gate	
WHY	A contract is useful only if easy to check	
DEPENDS	11.2.1-11.2.4	DELIVERABLE CLI validation command
ACCEPTANCE	Exit codes and human/JSON findings cover manifest, compatibility, schemas, permissions, health, and tests	

11.2.6	Build a minimal plugin template	Hard / 4h
GOAL	Scaffold manifest, one read-only tool, tests, docs, and CI	
WHY	Reduce contribution ambiguity	
DEPENDS	11.2.5	DELIVERABLE Versioned plugin template
ACCEPTANCE	Generated template validates and its fake integration test passes without external credentials	
11.2.7	Build one external example plugin	Hard / 6h
GOAL	Prove no core changes are needed	
WHY	This is the milestone completion test	
DEPENDS	11.2.2-11.2.6	DELIVERABLE Separately packaged example tool plugin
ACCEPTANCE	Install -> validate -> configure -> investigate -> uninstall works without modifying core source	
11.2.8	Add plugin compatibility matrix CI	Hard / 4h
GOAL	Test supported core/plugin version combinations	
WHY	Version ranges need evidence	
DEPENDS	11.1.5, 11.2.7	DELIVERABLE Matrix workflow
ACCEPTANCE	Supported combinations pass and intentionally incompatible combinations fail clearly	
11.2.9	Add malicious/broken-plugin tests	Hard / 5h
GOAL	Verify failure isolation and runtime safety	
WHY	Third-party code will fail or misbehave	
DEPENDS	11.2.2-11.2.4	DELIVERABLE Security/resilience suite
ACCEPTANCE	Invalid output, hangs, excessive output, secret-like data, permission mismatch, and crash are contained as specified	
11.3.1	Complete contributor documentation	Hard / 4h
GOAL	Explain setup, architecture, lifecycle, tools, scenarios, evaluation, testing, PRs, and releases	
WHY	External contributions need a coherent path	
DEPENDS	11.1.1-11.2.9	DELIVERABLE CONTRIBUTING.md and linked guides
ACCEPTANCE	A contributor can find the correct workflow from the repository root	
11.3.2	Add issue/PR templates and labels	Medium / 3h
GOAL	Standardize bugs, features, security, evaluation, areas, difficulty, and status	
WHY	Good issue hygiene lowers maintainer load	
DEPENDS	11.3.1	DELIVERABLE GitHub templates/label set
ACCEPTANCE	Templates request reproduction, evidence, scope, tests, and security impact where relevant	

11.3.3	Add Code of Conduct	Easy / 30m
GOAL	Set contribution behavior expectations	
WHY	Public collaboration needs explicit norms	
DEPENDS	None	DELIVERABLE CODE_OF_CONDUCT.md
ACCEPTANCE	Repository links the adopted standard and contact route	
11.3.4	Finalize Security Policy and disclosure path	Hard / 2h
GOAL	Give researchers a non-public reporting route and supported-version policy	
WHY	Public safety work needs responsible disclosure	
DEPENDS	11.1.6	DELIVERABLE SECURITY.md
ACCEPTANCE	Scope, report contents, response expectations, supported versions, and no-public-issue instruction are clear	
11.3.5	Define trademark/naming policy	Medium / 90m
GOAL	Clarify project-name/logo use if adoption grows	
WHY	License does not automatically grant trademark rights	
DEPENDS	1.1.2	DELIVERABLE Trademark note/policy
ACCEPTANCE	Permitted descriptive/community use and prohibited impersonation are explicit	
11.3.6	Publish a capability-based public roadmap	Medium / 2h
GOAL	Show reliable investigations/evaluation/actions/integrations instead of hype	
WHY	Contributors need honest direction	
DEPENDS	All prior phases	DELIVERABLE Public roadmap
ACCEPTANCE	Items have measurable exit criteria and exclude “multi-agent swarm”/integration-count theater	
11.3.7	Curate first contribution issues	Medium / 3h
GOAL	Create small scenario/docs/tool/test tasks with exact acceptance	
WHY	External contributors need bounded entry points	
DEPENDS	11.3.1-11.3.2	DELIVERABLE 5-10 ready issues
ACCEPTANCE	Each issue is reproducible, scoped, labeled, and does not require core redesign	
11.3.8	Run an external contributor pilot	Hard / 4-10h
GOAL	Have someone implement the example path from docs/template	
WHY	Real use exposes SDK friction better than self-review	
DEPENDS	11.2.6-11.3.7	DELIVERABLE Pilot notes/fixes
ACCEPTANCE	Contributor completes a bounded change without private guidance; blockers are fixed/documented	

11.3.9	Publish v0.8.0		Medium / 3h
GOAL	Release the stable initial plugin/community surface		
WHY	Marks extensibility based on exercised contracts		
DEPENDS	11.2.7-11.3.8	DELIVERABLE	Release, compatibility policy, template, example
ACCEPTANCE	External plugin adds a tool without core changes and passes the same safety/validation/audit path		

Phase 11 risks

Blocker or mistake	Typical beginner failure	Prevention
SDK stabilizes too early	Freezing speculative provider/policy/scenario APIs	Publish only multiply exercised surfaces; label the rest experimental
“Plugin validation” implies sandboxing	Treating schema checks as protection from malicious Python code	State trust assumptions honestly and run every tool through runtime controls
Compatibility becomes hand-waving	Using semantic versions without matrix tests/deprecation rules	Automate supported/incompatible combinations and document policy
Community work becomes cosmetic	Adding badges/templates without an external completion test	Require an independently packaged example and contributor pilot

Phase 11 Definition of Done

- Initial public plugin scope is narrow and based on exercised internal contracts.
- Manifest, loader, validator, health checks, template, compatibility matrix, and one external example work.
- Plugin tools cannot bypass schemas, grants, budgets, redaction, audit, policy, or action safety.
- Contributor/security/conduct/roadmap processes are discoverable and tested by an external pilot.
- A contributor adds a tool without core changes before v0.8.0.

Phase 12 - Release hardening, adoption proof, and 1.0 decision

Purpose: prove operational reliability, security, compatibility, documentation, and external usefulness before making stronger maturity claims. **Estimated phase effort:** 50-90 hours plus external feedback time. **Milestone:** production-readiness evidence package and an evidence-based 1.0/defer decision.

Learning prerequisites

Know: secure software supply chain, vulnerability management, load/resilience testing, backup/restore, upgrade compatibility, release candidates, incident response, user research. Safely defer: compliance certifications, enterprise sales features, global high availability, formal SLO commitments unsupported by operators.

Tasks

12.1.1	Define maturity claims and evidence	Hard / 3h
GOAL	List exactly what “reliable,” “secure,” “deployable,” and “production-ready” would require	
WHY	Prevent marketing language from outrunning proof	
DEPENDS	Phase 11	DELIVERABLE Release-readiness rubric
ACCEPTANCE	Every claim maps to a test, review, external observation, or is explicitly prohibited	
12.1.2	Refresh the complete threat model	Hard / 6h
GOAL	Re-evaluate tool, model, retrieval, action, API, worker, storage, integration, and plugin boundaries	
WHY	Threats evolve across phases	
DEPENDS	12.1.1	DELIVERABLE Consolidated threat model
ACCEPTANCE	All assets/trust boundaries/mitigations/residual risks are current and owners are assigned	
12.1.3	Perform permission inventory	Hard / 4h
GOAL	Enumerate every credential, scope, environment, and operation	
WHY	Least privilege can drift as integrations grow	
DEPENDS	12.1.2	DELIVERABLE Permission matrix
ACCEPTANCE	Every permission has a consumer/rationale/test; unused/broad grants are removed	
12.1.4	Run secret/redaction audit	Hard / 4h
GOAL	Inspect code, fixtures, traces, logs, errors, images, docs, and release artifacts	
WHY	Operational data and examples can leak credentials	
DEPENDS	12.1.2	DELIVERABLE Audit report/fixes
ACCEPTANCE	Secret scanning plus manual boundary review finds no real credential and adversarial tests remain green	

12.1.5	Add dependency vulnerability policy	Hard / 3h
GOAL	Define severity/response windows, supported versions, and exception process	
WHY	Continuous maintenance is part of security	
DEPENDS	11.3.4	DELIVERABLE Vulnerability policy/automation
ACCEPTANCE	Scans gate releases at defined severity and exceptions expire with owner/rationale	
12.1.6	Add SAST, container, IaC, and license checks	Hard / 5h
GOAL	Cover source and deployment artifacts	
WHY	Dependency scanning alone is incomplete	
DEPENDS	12.1.5	DELIVERABLE Layered security CI
ACCEPTANCE	Findings are triaged, false-positive exceptions are versioned, and high-severity release blockers are enforced	
12.1.7	Generate SBOMs and provenance	Hard / 4h
GOAL	Make release contents and build origin inspectable	
WHY	Supply-chain trust needs traceability	
DEPENDS	12.1.6	DELIVERABLE SBOM/provenance artifacts
ACCEPTANCE	Release artifacts have machine-readable component inventory tied to commit/build	
12.1.8	Sign release artifacts	Hard / 4h
GOAL	Allow consumers to verify authenticity	
WHY	Public binaries/images need integrity	
DEPENDS	12.1.7	DELIVERABLE Signing/verification workflow
ACCEPTANCE	Clean environment verifies signatures and documented failure cases	
12.1.9	Conduct independent security review	Hard / 8-20h
GOAL	Obtain a fresh review of highest-risk boundaries	
WHY	Author review misses assumptions	
DEPENDS	12.1.2-12.1.8	DELIVERABLE Findings, severity, remediation
ACCEPTANCE	Critical/high findings block release; medium residual risks are documented and owned	
12.2.1	Define workload/capacity targets	Hard / 3h
GOAL	Set realistic concurrent investigations, evidence sizes, scenario mix, and latency/cost budgets	
WHY	Load tests need product-relevant targets	
DEPENDS	10.6.6, 12.1.1	DELIVERABLE Capacity test plan
ACCEPTANCE	Targets reflect intended small-team use and distinguish provider/tool wait from platform overhead	

12.2.2	Run API/worker/database load tests	Hard / 8h
GOAL	Find saturation, queueing, locks, and memory growth	
WHY	Service behavior under concurrency must be measured	
DEPENDS	12.2.1	DELIVERABLE Load report/profiles
ACCEPTANCE	System meets or explicitly revises targets; no cross-investigation leakage or duplicate action occurs	
12.2.3	Test oversized/adversarial evidence	Hard / 5h
GOAL	Exercise logs, events, objects, traces, and documents at limits	
WHY	Context/output controls must hold under stress	
DEPENDS	7.1.4, 12.2.1	DELIVERABLE Limit/fuzz test report
ACCEPTANCE	Memory/time/context caps hold and failure is bounded/auditable	
12.2.4	Run dependency-failure drills	Hard / 8h
GOAL	Interrupt DB, provider, Kubernetes, GitHub, log, AWS, and worker dependencies	
WHY	Production-inspired reliability includes partial failure	
DEPENDS	10.6.7	DELIVERABLE Failure matrix
ACCEPTANCE	No fabricated completion/evidence, lost audit, unsafe retry, or duplicate action occurs	
12.2.5	Test restart/recovery and cancellation at every state	Hard / 6h
GOAL	Verify runtime durability across lifecycle boundaries	
WHY	State-machine correctness matters most during interruption	
DEPENDS	4.1.2-4.1.5, 10.4.4	DELIVERABLE Recovery test suite
ACCEPTANCE	Each state has defined resume/fail behavior and event replay remains consistent	
12.2.6	Implement and test backup/restore	Hard / 6h
GOAL	Protect PostgreSQL state, cases, policies, and audit records	
WHY	Persistent service needs recoverability	
DEPENDS	10.2.1-10.2.5	DELIVERABLE Backup/restore procedure
ACCEPTANCE	Restored environment passes integrity checks and replays representative investigations	
12.2.7	Test migration rollback/recovery	Hard / 5h
GOAL	Handle failed or incompatible schema upgrades	
WHY	Upgrade failure is operational risk	
DEPENDS	10.2.3, 12.2.6	DELIVERABLE Migration recovery playbook/tests
ACCEPTANCE	Supported upgrade path is verified from every supported release; failure has documented recovery	

12.2.8	Define operational indicators and alerts	Hard / 4h
GOAL	Identify service availability, queue, error, cost, safety, evaluation, and approval signals	
WHY	Operators need actionable health, not dashboards alone	
DEPENDS	10.6.4-10.6.5	DELIVERABLE Observability/alert specification
ACCEPTANCE	Each alert has symptom, threshold rationale, runbook, owner, and false-positive review	
12.2.9	Write platform runbooks	Hard / 5h
GOAL	Document provider outage, stuck worker, DB issue, runaway cost, failed action, and suspected security incident	
WHY	The operations product must itself be operable	
DEPENDS	12.2.4-12.2.8	DELIVERABLE Operator runbook set
ACCEPTANCE	Failure drills can be resolved or escalated using docs alone	
12.3.1	Freeze supported compatibility surface	Hard / 3h
GOAL	Declare supported CLI/API/schema/plugin/config/platform versions	
WHY	Users need explicit guarantees	
DEPENDS	11.1.5, 12.1.1	DELIVERABLE Compatibility matrix
ACCEPTANCE	Every supported combination has CI or a documented manual qualification	
12.3.2	Audit documentation against current behavior	Hard / 6h
GOAL	Remove stale commands, claims, diagrams, and screenshots	
WHY	Long projects accumulate dangerous documentation drift	
DEPENDS	All prior phases	DELIVERABLE Documentation audit/fixes
ACCEPTANCE	Link/command/schema examples pass automated checks where possible	
12.3.3	Run clean-room install/upgrade/uninstall tests	Hard / 8h
GOAL	Prove lifecycle docs on supported platforms	
WHY	Author machines hide dependencies	
DEPENDS	10.6.8, 12.3.1-12.3.2	DELIVERABLE Platform qualification report
ACCEPTANCE	New install, upgrade, rollback/recovery, and uninstall/cleanup succeed as documented	
12.3.4	Run the full benchmark on the release candidate	Hard / 12-24h elapsed, 4h hands-on
GOAL	Freeze quality, safety, cost, latency, and variance evidence	
WHY	Release should not rely on old baselines	
DEPENDS	6.4.5-6.5.3	DELIVERABLE RC evaluation report
ACCEPTANCE	No safety gate fails and regressions beyond policy are resolved or release is blocked	

12.3.5	Create end-to-end demonstration records	Hard / 5h
GOAL	Show failure, trigger, trace, evidence, diagnosis, action controls, and evaluation honestly	
WHY	Credibility comes from inspectable repeated work	
DEPENDS	12.3.4	DELIVERABLE Versioned demo docs/videos/traces
ACCEPTANCE	Demos use released code, disclose setup/limits, and are reproducible from text docs	

12.3.6	Verify legal/release files	Medium / 3h
GOAL	Confirm license, notices, third-party licenses, security, conduct, changelog, and trademark policy	
WHY	Public distribution needs complete metadata	
DEPENDS	12.1.5-12.1.8, 11.3.3-11.3.5	DELIVERABLE Release legal checklist
ACCEPTANCE	Automated license check and manual checklist pass	

12.4.1	Recruit at least one external user	Hard / variable
GOAL	Test the flagship workflow in an environment not built by the author	
WHY	Real usefulness outranks architecture claims	
DEPENDS	10.6.9, 11.3.8	DELIVERABLE User pilot plan
ACCEPTANCE	User understands data/permission risks and agrees on bounded non-production or controlled scope	

12.4.2	Observe external installation/investigation	Hard / 4-10h
GOAL	Capture blockers, confusion, incorrect assumptions, and actual value	
WHY	Self-reported “works” hides friction	
DEPENDS	12.4.1	DELIVERABLE Observation notes/issues
ACCEPTANCE	User completes or fails at documented steps without hidden author intervention; all blockers are recorded	

12.4.3	Prioritize pilot findings by product impact	Medium / 2h
GOAL	Fix reliability/safety/usability before feature requests	
WHY	External feedback can reopen scope indiscriminately	
DEPENDS	12.4.2	DELIVERABLE Prioritized pilot backlog
ACCEPTANCE	Each item has evidence, severity, target milestone, and acceptance criteria	

12.4.4	Complete one external contribution cycle	Hard / variable
GOAL	Validate issue -> branch/PR -> CI/review -> release-note workflow	
WHY	Open-source maturity needs more than repository visibility	
DEPENDS	11.3.7-11.3.8	DELIVERABLE Merged external contribution
ACCEPTANCE	Contributor completes the process and postmortem captures maintainership friction	

12.4.5	Review one-year outcome against original vision		Hard / 3h
GOAL	Compare supported domain, integrations, benchmark, docs, contributors/users, and limitations		
WHY	Decide based on evidence, not sunk cost or status		
DEPENDS	12.3.4, 12.4.2-12.4.4	DELIVERABLE	Outcome review
ACCEPTANCE	Every original success criterion is marked met/partial/unmet with linked evidence		
12.4.6	Make the 1.0 or defer decision		Hard / 2h
GOAL	Release 1.0 only if public contracts and operational evidence justify it		
WHY	Version numbers are trust signals		
DEPENDS	12.1.1, 12.4.5	DELIVERABLE	Decision ADR
ACCEPTANCE	Decision cites compatibility, security, evaluation, external use, and unresolved risks		
12.4.7	Publish the qualified release		Hard / 4h
GOAL	Ship 1.0.0 or a truthful later 0.x hardening release		
WHY	Do not force maturity to match the calendar		
DEPENDS	12.4.6	DELIVERABLE	Signed release, artifacts, SBOM, changelog, benchmark, limits
ACCEPTANCE	All chosen release gates pass and claims match the evidence package		
12.4.8	Create the post-release maintenance cadence		Medium / 2h
GOAL	Define dependency/security review, benchmark, issue triage, release, and support rhythms		
WHY	Reliability decays without maintenance		
DEPENDS	12.4.7	DELIVERABLE	Maintainer runbook/calendar
ACCEPTANCE	Owners/frequency/triggers exist for every recurring responsibility		

Phase 12 risks

Blocker or mistake	Typical beginner failure	Prevention
"Production-ready" is declared from architecture	Equating Kubernetes deployment and security tooling with operational proof	Use the explicit evidence rubric and external pilot
Hardening becomes infinite	Adding enterprise features without a defined claim/gate	Tie every task to a release claim or measured failure
Release pressure hides regressions	Relaxing thresholds to hit 1.0	Let the decision be "defer"; never average away safety failures
External feedback causes random scope growth	Implementing every requested integration	Prioritize repeatability, safety, usability, and flagship depth first

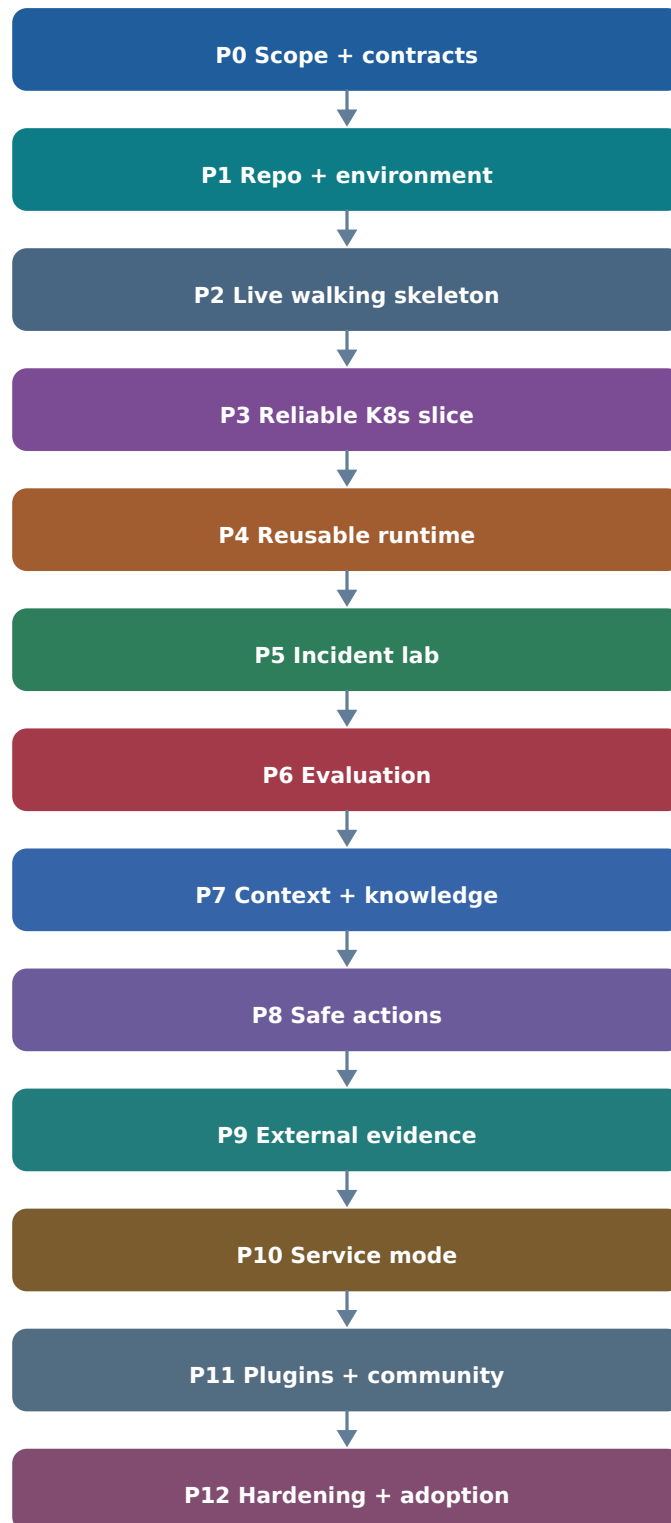
Phase 12 Definition of Done

- Security, permissions, supply chain, load, failure recovery, backup/restore, migrations, and operations are tested and documented.
- A current release-candidate benchmark passes hard safety gates and reports quality/cost/latency variance.
- Clean-room install/upgrade/cleanup works on supported environments.

- At least one external user attempts the flagship workflow and one external contribution completes the project process.
 - The original success criteria are reviewed against evidence.
 - The project publishes either a qualified 1.0.0 or an honest 0.x release with a documented reason to defer 1.0.
-

5. Critical path and dependency graph

Primary dependency graph



This is intentionally close to the order in the source specification. Later work may be prepared early, but its public interface or completion claim must wait for the gate immediately before it.

Critical-path gates

Gate	Must be true	Blocks
CP-1 Scope gate	One scenario, input/output schemas, read-only boundary, and non-goals are explicit	Repository implementation choices and acceptance tests
CP-2 First-run gate	Real Kubernetes evidence reaches one validated evidence-linked model diagnosis	Any claim that the product works
CP-3 M1 reliability gate	Three scenarios pass repeated live runs without prompt editing	Runtime generalization
CP-4 Runtime gate	Explicit state machine, shared tool contract, provider boundary, persistence, and replay work	Incident-lab scale and credible extension claims
CP-5 Lab gate	Scenarios are ground-truthed, isolated, automatically resettable, and contract-validated	Evaluation validity
CP-6 Evaluation gate	Tested scorers, repeated baseline, hard safety gates, and comparison reports work	Context, safety, or integration “improvement” claims
CP-7 Context gate	Retrieval/compression features show measured value and preserved safety	Keeping those features and adding action reasoning context
CP-8 Action-safety gate	No mutation bypasses proposal, policy, approval, preconditions, audit, and verification	Public remediation capability
CP-9 Cross-system gate	One scenario proves correlated external evidence improves results	Service-mode stabilization around a meaningful workflow
CP-10 External-user gate	A clean user installs and completes the workflow	Plugin/API stabilization claim
CP-11 Extension gate	A separately packaged tool is added without core changes and passes safety controls	Public plugin maturity claim
CP-12 Release gate	Security/reliability/benchmark/upgrade/external-use evidence passes	1.0 or “production-ready” language

Non-negotiable ordering constraints

- Do not build the generic runtime before the Day-0 path exists; otherwise abstractions are speculative.
- Do not build the evaluation scorer before scenarios have known ground truth and repeatable lifecycle.
- Do not keep context/retrieval features that have no measured benefit.
- Do not implement mutating tools before deterministic policy, approvals, and audit contracts exist.
- Do not add service/web mode before the local CLI workflow is deep and reliable.
- Do not publish plugin contracts before built-in implementations have exercised them.
- Do not claim 1.0 because a year passed.

6. Parallel work map

For a solo maintainer, “parallel” means independent preparation or separate short-lived branches-not five simultaneous unfinished features. Keep one behavior-changing branch active and use the other track for documents, fixtures, or review while tests/builds run.

Primary work	Safe parallel work	Earliest start	Merge rule
Phase 0 schemas	Scenario prose, non-goals, threat inventory	After 0.1.3	Resolve names against schemas before implementation starts
Phase 1 environment setup	README/ADR/docs structure	After repository creation	Never commit machine-specific config or secrets
Phase 2 Kubernetes collectors	Fake-provider and fixture design	After EvidenceRecord/ToolResult are frozen	Integrate only after both sides pass unit tests
Phase 3 new scenarios	Collector robustness and CI	After scenario template and current regression baseline	One scenario per PR; all existing scenarios must stay green
Phase 4 tool registry	Provider interface and SQLite repository	After the state/event model is fixed	Merge behind contract tests; migrate one integration at a time
Phase 5 individual scenarios	Scenario validator/docs	After ScenarioDefinition v1	Random-order full lab check before milestone
Phase 6 individual scorers	Report UI/format and runner resilience	After metric semantics are fixed	Golden scorer suite gates every merge
Phase 7 runbook retrieval	Evidence transformers and context debug view	After ContextPackage v1	Use separate ablation labels; do not change several context dimensions in one experiment
Phase 8 policy engine	Approval records and action proposal UI/CLI design	After ActionProposal/policy contracts	No mutating adapter merges until end-to-end controls exist
Phase 9 GitHub adapter	Log-backend decision/lab setup; AWS fixtures	After adapter/credential boundary	Each integration must pass offline contracts before live tests
Phase 10 API	PostgreSQL backend and worker design; CLI client	After service resource/auth specs	Contract/schema compatibility and authorization tests gate integration
Phase 10 web views	Container/deployment/telemetry work	After API contracts	UI never becomes the only way to exercise a capability
Phase 11 plugin template/docs	Loader/validator security tests	After manifest v1	External example must install against released/public interfaces only
Phase 12 security review	Clean-room docs test, external pilot scheduling, performance test prep	After release-candidate scope freeze	Critical/high findings and benchmark safety failures block release

7. Milestones and timeline

Milestone definitions

Milestone	User-visible checkpoint	Exit proof	Suggested release
M0	Architecture foundation	First workflow explained on one page with contracts, threat boundary, and non-goals	None
M0.5	First working investigation	Live broken-readiness run with evidence-linked validated report	v0.0.1 optional
M1	MVP complete	Three repeatable read-only Kubernetes diagnoses, deterministic CI, trace replay	v0.1.0
M2	Platform core	Two independent tool packages use one explicit runtime; local persistence/replay	v0.2.0
M3	Reproducible incident lab	Seven setup/fault/verify/reset scenarios with ground truth	Included with v0.3.0
M4	Evaluation differentiator	Repeated benchmark, tested scorers, hard safety gates, comparisons	v0.3.0
M5	Context/knowledge	Runbooks/cases improve benchmarked outcomes without unacceptable regressions	v0.4.0
M6	Safe remediation	Three dev-only actions pass policy/approval/audit/verification	v0.5.0
M7	Cross-system investigation	Deployment change is correlated with Kubernetes/log/external evidence	v0.6.0
M8	Deployable service	New user deploys and runs the flagship workflow from docs alone	v0.7.0
M9	Community extension	External plugin adds a tool without core edits; contributor pilot succeeds	v0.8.0
M10	Qualified maturity	Security/reliability/upgrade/benchmark/external-use evidence supports a maturity decision	v1.0.0 or honest later 0.x

Realistic timeline

The complete WBS is approximately **526-845 focused hours**, plus waiting for external feedback. That means:

- At 20 focused hours/week: roughly 6-10 months.
- At 15 focused hours/week: roughly 8-13 months.
- At 10-12 focused hours/week: roughly 11-20 months.

Six months is not realistic at 10-15 hours/week unless Phases 9-12 are cut or heavily reduced. Codex accelerates implementation, but it does not remove review, debugging, learning, evaluation runs, or security work.

Target window at ~15-20 h/week	Milestone	Cumulative outcome
Day 1	M0.5	One narrow live diagnosis; not a finished MVP
Weeks 1-5	M1	Credible first portfolio/demo release
Weeks 6-10	M2	Reusable runtime rather than one-off flow
Weeks 11-16	M3	Stable scenario laboratory
Weeks 17-22	M4	Measurable AI engineering and v0.3.0
Weeks 23-27	M5	Evaluated context and operational knowledge
Weeks 28-33	M6	Bounded development remediation
Weeks 34-40	M7	Deep cross-system workflow
Weeks 41-48	M8	Deployable service used from docs
Weeks 49-53	M9	Plugin and contributor proof
Weeks 54+	M10	Hardening, pilots, and evidence-based 1.0 decision

Scope-cut option for a strong 6-month portfolio project

Stop after M4, then do only the most valuable slice of M5 and M7. A project with a reliable Kubernetes investigator, seven reproducible scenarios, and a serious evaluation suite is already stronger than a sprawling half-built service/plugin platform. Phases 8-12 are year-one product expansion, not prerequisites for resume value.

8. Git commit strategy

Commit rules

1. Use one logical behavior or document decision per commit.
2. Commit tests with the behavior they verify; do not create a later “add tests” dump unless tests truly could not exist earlier.
3. Keep formatting-only changes separate from behavior changes.
4. Never mix generated dependency-lock changes with unrelated refactors without explaining why.
5. Do not commit secrets, kubeconfig, raw sensitive traces, local DBs, coverage output, model caches, or environment files.
6. Run the narrow checks before each commit and the full deterministic gate before each push/PR.
7. The user-not Codex-reviews, stages, commits, and pushes.
8. A commit message describes the result, not the activity: `feat(k8s): collect bounded workload events`, not `work on events`.

Recommended Conventional Commit types

Type	Use
feat	New user/runtime capability
fix	Correct observable behavior or safety defect
test	Test-only improvement with no production behavior change
docs	Documentation-only change
refactor	Internal structure with deliberately unchanged behavior
chore	Tooling, dependencies, repository maintenance
ci	CI workflow/check changes
perf	Measured performance/cost improvement
security	Optional project-specific type for hardening; otherwise use <code>fix(security)</code>

Suggested Day-0 commit sequence

1. `chore(repo): initialize project foundation`
2. `docs(architecture): define first read-only investigation`
3. `feat(scenarios): add broken readiness lab`
4. `feat(k8s): collect bounded deployment evidence`
5. `feat(agent): produce validated evidence-linked diagnosis`
6. `test(investigation): cover fixture-backed end-to-end flow`
7. `docs(quickstart): document first live investigation`

Do not force seven commits if a change is too small to stand alone. The rule is reviewability, not commit-count theater.

9. Branch strategy

Use **trunk-based development with short-lived branches**.

- `main` is always releasable and protected by required deterministic checks.
 - Do not create a long-lived `develop` branch.
 - Branch from current `main`; rebase/update before final review.
 - Keep one capability per branch and aim to merge within one to three focused days.
 - Use names such as `feat/day0-broken-readiness`, `feat/tool-registry`, `fix/evidence-id-validation`, `docs/scenario-authoring`, and `security/action-preconditions`.
 - Open a PR even when working solo once CI exists; use it as the review/audit boundary.
 - Prefer squash merge for small single-purpose PRs. Keep a multi-commit merge only when the individual commits are independently meaningful and clean.
 - Delete merged branches.
 - Tag releases from protected `main` only after the milestone gate passes.
 - Use release tags `v0.1.0`, `v0.2.0`, and so on; patch releases contain fixes, not new milestone scope.
 - For emergency security fixes, use `security/<short-name>` from the supported release/main line, add regression tests, and document affected versions.
-

10. Documentation and planning structure

Create directories only when their first real file is needed. The eventual structure should converge on:

```

README.md
LICENSE
CHANGELOG.md
CONTRIBUTING.md
CODE_OF_CONDUCT.md
SECURITY.md
docs/
  product/
    charter.md
    non-goals.md
  user/
    quick-start.md
    first-investigation.md
    configuration.md
    integrations.md
    limitations.md
  architecture/
    overview.md
    investigation-lifecycle.md
    state-model.md
    tool-contract.md
    model-interface.md
    context-engine.md
    persistence.md
  adr/
    0001-language-and-packaging.md
    0002-monorepo.md
    0003-cli-first.md
  security/
    threat-model.md
    authorization-model.md
    policy-model.md
    action-safety.md
  scenarios/
    format.md
    broken-readiness.md
    missing-secret.md
    image-pull-failure.md
  evaluations/
    methodology.md
    metrics.md
    baselines/
  contributing/
    local-development.md
    tool-authoring.md
    scenario-authoring.md
    evaluation-authoring.md
    plugin-authoring.md
  demonstrations/
    broken-readiness.md
  operations/
    deployment.md
    backup-restore.md
    runbooks/
  planning/
    roadmap.md
    deferred.md
    release-readiness.md
  releases/
    release-process.md

```

Planning-file rules:

- `roadmap.md` states capability order and exit gates; it is not a daily diary.
- GitHub Issues hold bounded executable work once the repository is public.
- `deferred.md` catches tempting future work so it does not enter the current branch.
- ADRs record durable technical decisions; do not use a giant mutable “decisions” file.

- The current implementation and tests remain the source of truth for implementation reality.
 - Update documentation in the same PR as behavior when the behavior changes.
-

11. Daily execution mode

Standard 90-120 minute session

- Select - 5 minutes:** choose the lowest-numbered unchecked leaf task whose dependencies are complete.
- Bound - 5 minutes:** write a Codex request with one goal, files/area to inspect, non-goals, acceptance criteria, and required tests.
- Implement - 45-75 minutes:** let Codex inspect and change the repository; answer only blockers that materially alter the task.
- Review - 15-25 minutes:** inspect the diff, understand the data/control flow, remove unrelated changes, and check safety boundaries.
- Verify - 10-20 minutes:** run the narrow tests, then relevant lint/type/integration checks.
- Close - 5 minutes:** update the checklist/issue, commit, push/open PR if ready, and record the next unblocked WBS ID.

If the task is not done, do not start another. End with a clear checkpoint and the exact failing test/blocker. If a task repeatedly exceeds two hours, split it into contract, implementation, negative tests, integration, and documentation sessions.

Canonical session queue

The WBS leaf order is the canonical queue. The table below budgets sessions; within each range, take the next unblocked leaf task in numerical order.

Session range	WBS scope	Approx. sessions	Exit output
S001-S004	Phase 0	4	Scope/contracts/threat model/ADRs
S005-S010	Phase 1	6	Public reproducible repository/toolchain
S011-S017	Phase 2	7	First live evidence-linked diagnosis
S018-S042	Phase 3	25	v0.1.0, three repeatable scenarios
S043-S073	Phase 4	31	v0.2.0, reusable runtime
S074-S108	Phase 5	35	Seven-scenario incident lab
S109-S141	Phase 6	33	v0.3.0, benchmark/evaluation
S142-S168	Phase 7	27	v0.4.0, measured context/knowledge
S169-S204	Phase 8	36	v0.5.0, controlled dev actions
S205-S244	Phase 9	40	v0.6.0, cross-system workflow
S245-S295	Phase 10	51	v0.7.0, deployable service
S296-S325	Phase 11	30	v0.8.0, external plugin/contributor path
S326-S365	Phase 12	40+	Qualified maturity release or documented defer

The session numbers are capacity buckets, not deadlines. External evaluations, benchmark repetitions, security review, and user pilots can add elapsed time without adding hands-on sessions.

Session Definition of Done

A session is closed only when one of these is true:

- The leaf task's deliverable and acceptance criteria are satisfied, verified, and committed; or
- A real external blocker is documented with reproduction, evidence, impact, attempted fixes, and the exact next action.

"Codex produced code," "it looks right," "the model diagnosed it once," and "I read about the technology" are not completed sessions.

12. First week - immediate checklist

Today: minimum working version

This is a **single rested 7-10 hour build day** if Docker/Python/GitHub authentication mostly work. If starting at the end of a night shift, do Sessions T1-T2, sleep, then continue; tired review is where secrets and unsafe shortcuts get accepted.

T1 - Repository baseline (45-60m)

- [] Complete 1.1.1-1.1.7.
- [] Use the working name `ai-ops-control-plane`; do not spend the day branding.
- [] Add the minimal README/license/ignore files.
- [] Commit: `chore(repo): initialize project foundation`.
- **Output:** public empty-but-valid project baseline.

T2 - One-page scope and contracts (45-60m)

- [] Complete lean versions of 0.1.1-0.1.7.
- [] Complete 0.2.2-0.2.6 and 0.2.8 at v0 draft depth.
- [] Write the non-goals before asking Codex for implementation.
- [] Commit: `docs(architecture): define first read-only investigation`.
- **Output:** one page explaining intake, collectors, evidence, model diagnosis, validation, report, and read-only boundary.

T3 - Toolchain scaffold (45-75m, plus installation time)

- [] Complete 1.2.1-1.2.9.
- [] Complete 1.3.1-1.3.4.
- [] Verify the default deterministic quality commands run.
- [] Commit: `chore(tooling): configure Python quality and test environment`.
- **Output:** installable package with locked dependencies and no product behavior yet.

T4 - Broken-readiness live lab (60-90m)

- [] Complete 2.1.1-2.1.3.
- [] Complete 2.2.1-2.2.7.
- [] Manually prove the healthy baseline and single injected fault.
- [] Verify read permissions work and write permissions fail.
- [] Commit: `feat(scenarios): add broken readiness lab`.
- **Output:** disposable kind scenario with known ground truth.

T5 - Evidence collection (90-120m)

- [] Complete 2.3.1-2.3.8.
- [] Implement only Deployment/Pod status, recent events, and recent logs.
- [] Add fake/fixture tests as each collector is completed.

- [] Do not add a tool registry or agent framework.
- [] Commit: `feat(k8s): collect bounded deployment evidence`.
- **Output:** a deterministic evidence package from the live target.

T6 - Structured diagnosis and CLI (90-120m)

- [] Complete 2.3.9-2.3.10.
- [] Complete 2.4.1-2.4.7.
- [] Use one provider call and independently validate evidence IDs.
- [] Do not allow model-selected shell or Kubernetes mutation.
- [] Commit: `feat(agent): produce validated evidence-linked diagnosis`.
- **Output:** end-to-end CLI using a fake provider, then the real provider.

T7 - Prove and checkpoint (60-90m)

- [] Complete 2.5.1-2.5.9 except any nice-to-have trace presentation.
- [] Run deterministic fake-provider E2E.
- [] Run live evidence collection without model.
- [] Run the full live investigation once, then rerun it at least twice without prompt edits.
- [] Save a sanitized trace and write exact quick-start/reset steps.
- [] Review the diff for secret exposure, write permissions, shell execution, and unused architecture.
- [] Merge and optionally tag `v0.0.1`.
- **Output:** smallest credible first working version.

Three-hour fallback floor

If Docker/kind or provider access blocks the live target, do not fabricate completion. Finish the same CLI path against a captured/synthetic Kubernetes fixture and fake provider, document the environment blocker, and make the first next session the live adapter. This is a walking skeleton checkpoint, not M0.5 and not an MVP.

Days 2-7

Day 2 - Finish foundation debt

- [] Complete remaining Phase 0 tasks, especially 0.2.7 and 0.3.1-0.3.4.
- [] Complete remaining Phase 1 documentation/settings tasks.
- [] Review today's code and explain every module without Codex.
- [] Convert discovered defects into bounded GitHub Issues.
- **Expected commit(s):** ADR/threat-model docs and narrow fixes only.

Day 3 - Freeze regression behavior

- [] Complete 3.1.1-3.1.2.
- [] Expand fixture tests for forbidden, not-found, no logs, truncation, and malformed provider output.
- [] Record timeout, retry, output-limit, and redaction gaps as bounded follow-up work; do not mark 3.4.1-3.4.4 complete before their dependencies.

- **Expected output:** the Day-0 regression baseline and formal collector input contracts.

Day 4 - Add rollout evidence

- [] Complete 3.1.3-3.1.6.
- [] Confirm the broken-readiness scenario still passes.
- [] Save a before/after context snapshot and verify useful fields were preserved.
- **Expected commit:** feat(k8s): add normalized rollout and object evidence.

Day 5 - Define and deploy metric evidence

- [] Complete 3.1.7-3.1.8.
- [] Keep the metric catalog limited to questions used by the flagship investigation.
- [] Verify every supported question returns data or an explicit documented unavailable state.
- **Expected commit:** feat(lab): add bounded workload metrics.

Day 6 - Normalize Prometheus evidence

- [] Complete 3.1.9-3.1.10.
- [] Prove query allow-listing, time/output caps, label handling, and missing/stale-data behavior.
- [] Rerun broken-readiness to prove the new evidence path does not break the Day-0 result.
- **Expected commit:** feat(prometheus): collect bounded metric evidence.

Day 7 - Start explicit investigation state

- [] Complete 3.2.1-3.2.3.
- [] Complete 1.3.7 if starter CI was not already added.
- [] Run format, lint, type, unit, fake-E2E, schema, and secret checks from a clean shell.
- [] Update README with only capabilities that actually pass.
- [] Record evidence quality, calls, latency, cost, and failures for all live attempts.
- [] Choose the first unblocked WBS task for Week 2; default is 3.2.4, not a new feature idea.
- **Expected output:** green baseline, versioned planning/hypothesis/state schemas, and an honest Week 1 status report.

Week 1 stop list

Do not spend Week 1 on:

- a project logo or perfect name;
- a web UI or API;
- PostgreSQL or a vector database;
- LangGraph/CrewAI/multi-agent architecture merely because it is available;
- AWS, GitHub Actions evidence, Loki/OpenSearch, or Argo CD integrations;
- Kubernetes deployment of the agent itself;
- plugin architecture;
- automatic rollback/restart;
- comprehensive prompt-engineering infrastructure;

- more roadmap analysis after the T2 scope page is complete.

13. Ready-to-use Codex task packets for today

Use these sequentially. Let Codex inspect the actual repository before every change. You review and commit manually.

Packet A - Repository and minimal package foundation

Inspect the repository first. Initialize only the minimum Python 3.12+ uv project needed for a CLI-based AI operations investigation. Configure pytest, Ruff, and mypy; add a safe `.gitignore`, `.env.example`, and the minimal source/test directories. Do not implement product behavior, create future modules, commit, or push. Preserve existing files. Run the deterministic setup/quality checks and report changed files, commands run, and any blocker.

Packet B - Reproducible broken-readiness scenario

Inspect the repository and the scenario/architecture documents first. Add one local kind scenario containing a healthy baseline and a single broken-readiness fault. Provide deterministic setup, fault, verify, reset, and teardown paths plus a namespace-scoped read-only identity. Do not add Prometheus, a demo platform, Helm, cloud resources, or application features unrelated to the fault. Verify healthy-before-fault, expected unhealthy evidence after fault, successful reset, and denied write permission. Do not commit or push. Report the exact verification results.

Packet C - Typed read-only evidence collection

Inspect the current repository and contracts first. Implement the smallest typed evidence path for a named Kubernetes Deployment: configuration loading, EvidenceRecord, ToolResult, FinalDiagnosis schemas, explicit-context Kubernetes client construction, Deployment/Pod inspection, relevant recent events, and bounded recent logs. Use the official Kubernetes client; no arbitrary shell or model-generated `kubectl`. Normalize, time-bound, size-limit, sanitize, and test outputs. Add fixtures so default tests need no cluster. Do not add a generic tool registry, planner framework, persistence, API, or mutation. Do not commit or push. Run format, lint, type, and relevant tests and report results.

Packet D - One structured diagnosis and CLI

Inspect the existing collectors and contracts first. Add one thin model-provider boundary with a deterministic fake and one configured live provider. Build a bounded context package from normalized evidence, request one schema-valid diagnosis, independently reject unknown evidence IDs, and render human-readable plus JSON CLI output. The model may diagnose and recommend only; it may not select arbitrary commands or execute Kubernetes actions. Handle missing config, auth, target, tool, provider, and schema failures distinctly. Add a fake-provider end-to-end test. Do not introduce an open-ended agent loop, framework, database, API, or future abstractions. Do not commit or push. Run the full deterministic quality gate and report the diff and results.

Packet E - Live verification and scope audit

Inspect the complete Day-0 path. Run deterministic checks, then verify live evidence collection against the broken-readiness kind scenario, then run the configured live model diagnosis. Fix only defects that prevent the written Day-0 acceptance criteria. Save a sanitized replayable trace and complete the quick start. Audit for committed secrets, implicit non-local kube contexts, write-capable Kubernetes calls, arbitrary shell execution, raw oversized objects/logs, fabricated evidence references, and unused future architecture. Do not broaden scope, commit, or push. Report each acceptance criterion as pass/fail with evidence and list any remaining blocker.

14. Global Definition of Done

A capability is done only when:

- Its user-visible behavior works through the real intended boundary, not only a mocked internal function.
- Inputs, outputs, errors, permissions, budgets, sensitivity, and audit behavior are explicit.
- Deterministic positive and negative tests pass; live tests are separated and reproducible where required.
- The behavior survives relevant existing scenarios/evaluations without unacceptable regression.
- Secrets and sensitive tool output do not enter prompts, traces, logs, fixtures, or Git unnecessarily.
- Documentation, examples, schemas, ADRs, and compatibility notes match the implementation.
- CI is green, the diff is understood by the maintainer, and the change is committed on a bounded branch/PR.
- Limitations and unresolved risks are stated honestly.

The project is not done when it has many components. It is done when a bounded operational investigation works repeatedly, explains its evidence, behaves safely under failure, and can be reproduced by someone other than its author.